Cloud Data Engineer - Jain University

Name : Twinkle Meska

USN : 23BTRSN051

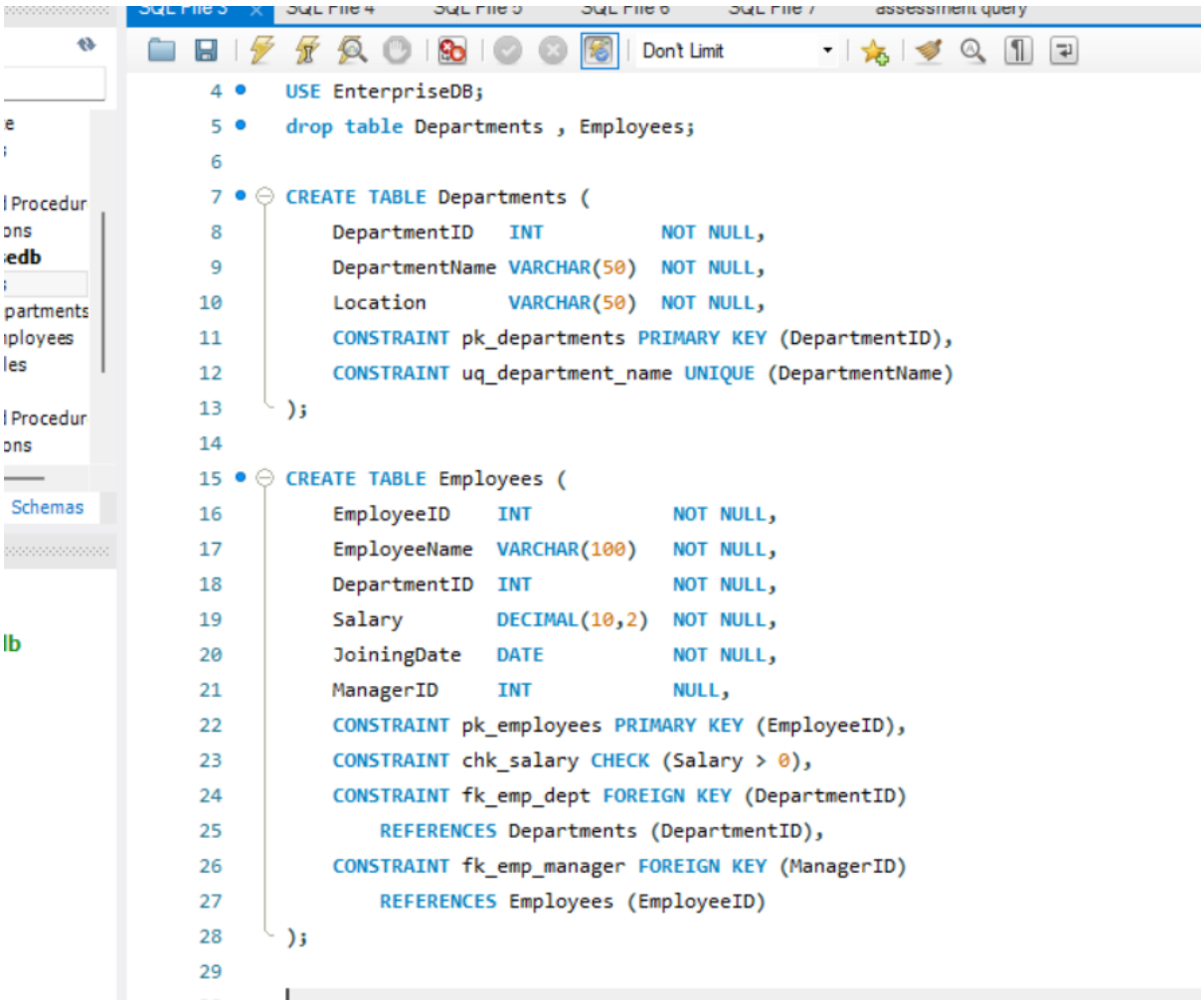
HCLID : HCLTFP2260389

College Name: Jain University

Batch: 2027

SECTION A: ADVANCED SQL || Maximum Marks: 20

Question 1: Database Design and Advanced Queries (8 Marks) Create a database named EnterpriseDB. Create the following tables

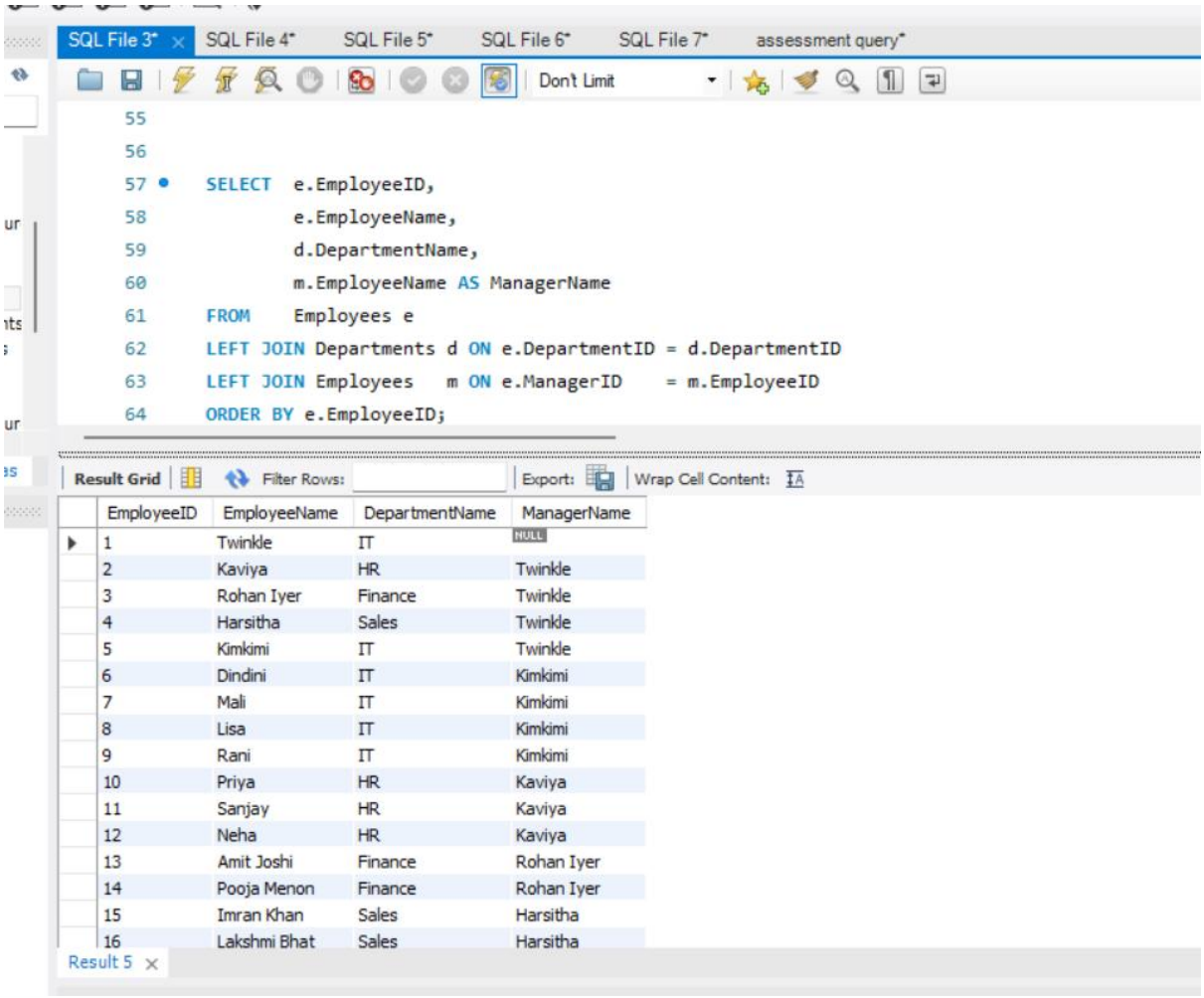


```sql
USE EnterpriseDB;
drop table Departments , Employees;

CREATE TABLE Departments (
    DepartmentID   INT          NOT NULL,
    DepartmentName VARCHAR(50)  NOT NULL,
    Location       VARCHAR(50)  NOT NULL,
    CONSTRAINT pk_departments PRIMARY KEY (DepartmentID),
    CONSTRAINT uq_department_name UNIQUE (DepartmentName)
);

CREATE TABLE Employees (
    EmployeeID    INT            NOT NULL,
    EmployeeName  VARCHAR(100)   NOT NULL,
    DepartmentID  INT            NOT NULL,
    Salary        DECIMAL(10,2)  NOT NULL,
    JoiningDate   DATE           NOT NULL,
    ManagerID     INT            NULL,
    CONSTRAINT pk_employees PRIMARY KEY (EmployeeID),
    CONSTRAINT chk_salary CHECK (Salary > 0),
    CONSTRAINT fk_emp_dept FOREIGN KEY (DepartmentID)
        REFERENCES Departments (DepartmentID),
    CONSTRAINT fk_emp_manager FOREIGN KEY (ManagerID)
        REFERENCES Employees (EmployeeID)
);
```

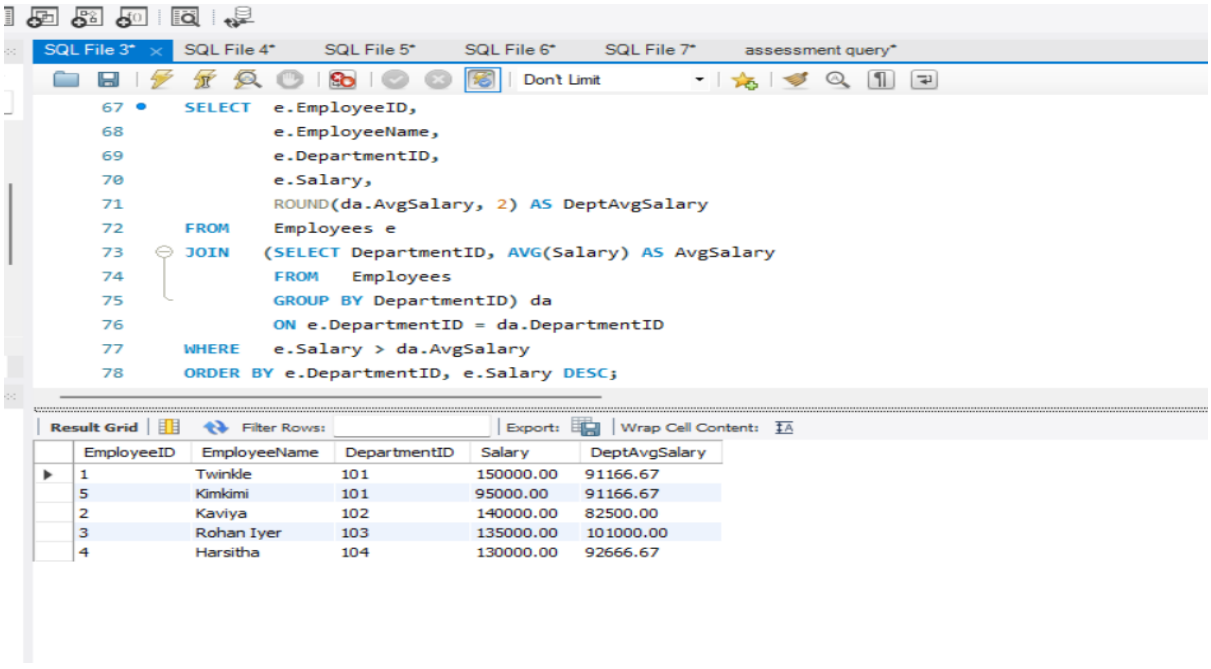
a. Display employee names, department names, and manager names using joins. (2 marks)



```
SELECT  e.EmployeeID,
        e.EmployeeName,
        d.DepartmentName,
        m.EmployeeName AS ManagerName
FROM    Employees e
LEFT JOIN Departments d ON e.DepartmentID = d.DepartmentID
LEFT JOIN Employees   m ON e.ManagerID    = m.EmployeeID
ORDER BY e.EmployeeID;
```

EmployeeID	EmployeeName	DepartmentName	ManagerName
1	Twinkle	IT	NULL
2	Kaviya	HR	Twinkle
3	Rohan Iyer	Finance	Twinkle
4	Harsitha	Sales	Twinkle
5	Kimkimi	IT	Twinkle
6	Dindini	IT	Kimkimi
7	Mali	IT	Kimkimi
8	Lisa	IT	Kimkimi
9	Rani	IT	Kimkimi
10	Priya	HR	Kaviya
11	Sanjay	HR	Kaviya
12	Neha	HR	Kaviya
13	Amit Joshi	Finance	Rohan Iyer
14	Pooja Menon	Finance	Rohan Iyer
15	Imran Khan	Sales	Harsitha
16	Lakshmi Bhat	Sales	Harsitha

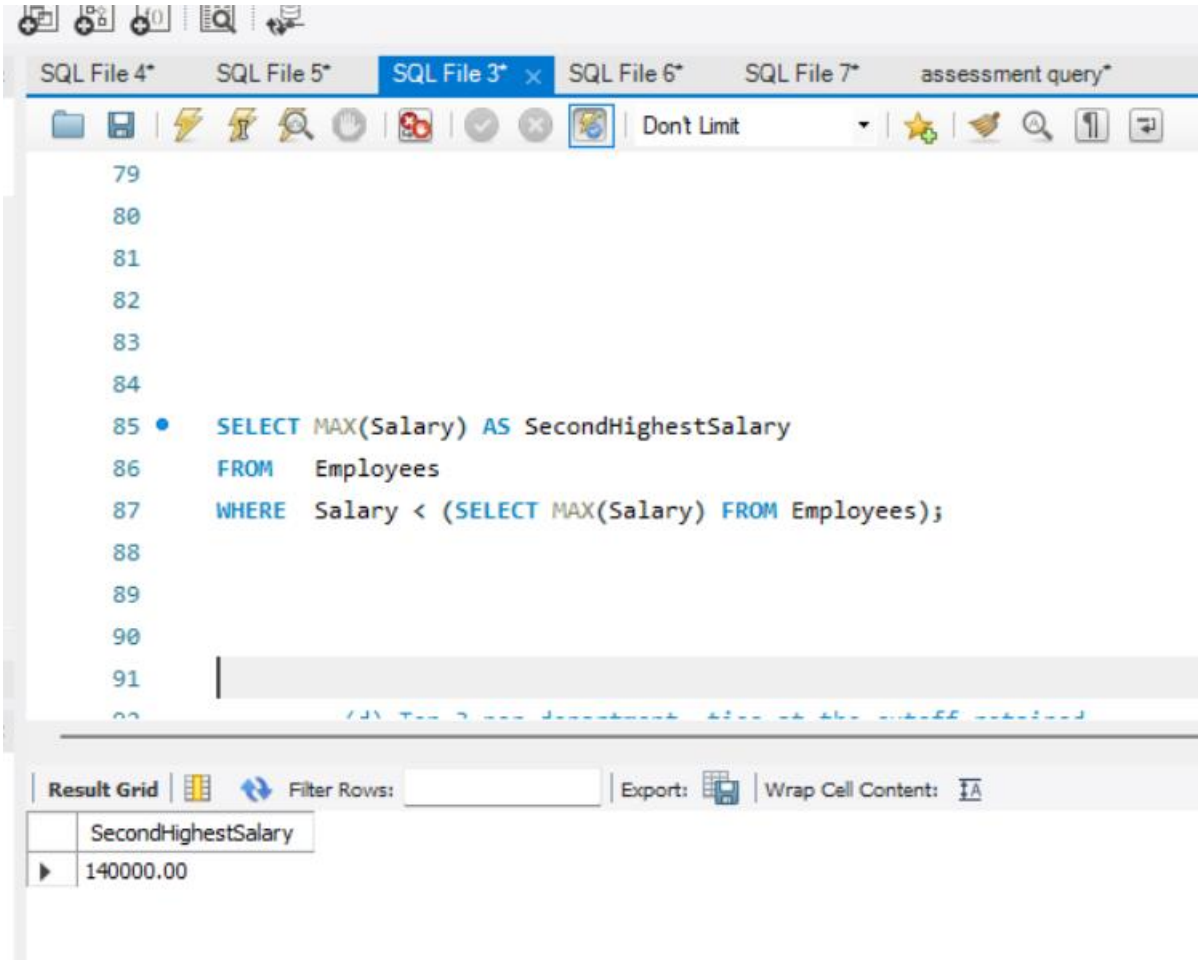
b. Find employees earning more than their department's average salary. (2 marks)



```
SELECT  e.EmployeeID,
        e.EmployeeName,
        e.DepartmentID,
        e.Salary,
        ROUND(da.AvgSalary, 2) AS DeptAvgSalary
FROM    Employees e
JOIN   (SELECT DepartmentID, AVG(Salary) AS AvgSalary
        FROM   Employees
        GROUP BY DepartmentID) da
        ON e.DepartmentID = da.DepartmentID
WHERE   e.Salary > da.AvgSalary
ORDER BY e.DepartmentID, e.Salary DESC;
```

EmployeeID	EmployeeName	DepartmentID	Salary	DeptAvgSalary
1	Twinkle	101	150000.00	91166.67
5	Kimkimi	101	95000.00	91166.67
2	Kaviya	102	140000.00	82500.00
3	Rohan Iyer	103	135000.00	101000.00
4	Harsitha	104	130000.00	92666.67

c. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST. (2 marks)



d. Rank employees by salary within each department and display the top three employee per department, retaining ties at the cutoff. (2 marks)

SQL File 4* | SQL File 5* | SQL File 3* | SQL File 6* | SQL File 7* | assessment query*

Don't Limit

```
93  SELECT  DepartmentName, EmployeeID, EmployeeName, Salary, SalaryRank
94  FROM (
95          SELECT  d.DepartmentName,
96                  e.EmployeeID,
97                  e.EmployeeName,
98                  e.Salary,
99                  RANK() OVER (PARTITION BY e.DepartmentID
100                              ORDER BY e.Salary DESC) AS SalaryRank
101         FROM    Employees e
102         JOIN    Departments d ON e.DepartmentID = d.DepartmentID
103     ) ranked
104 WHERE   SalaryRank <= 3
105 ORDER BY DepartmentName, SalaryRank, EmployeeName;
```

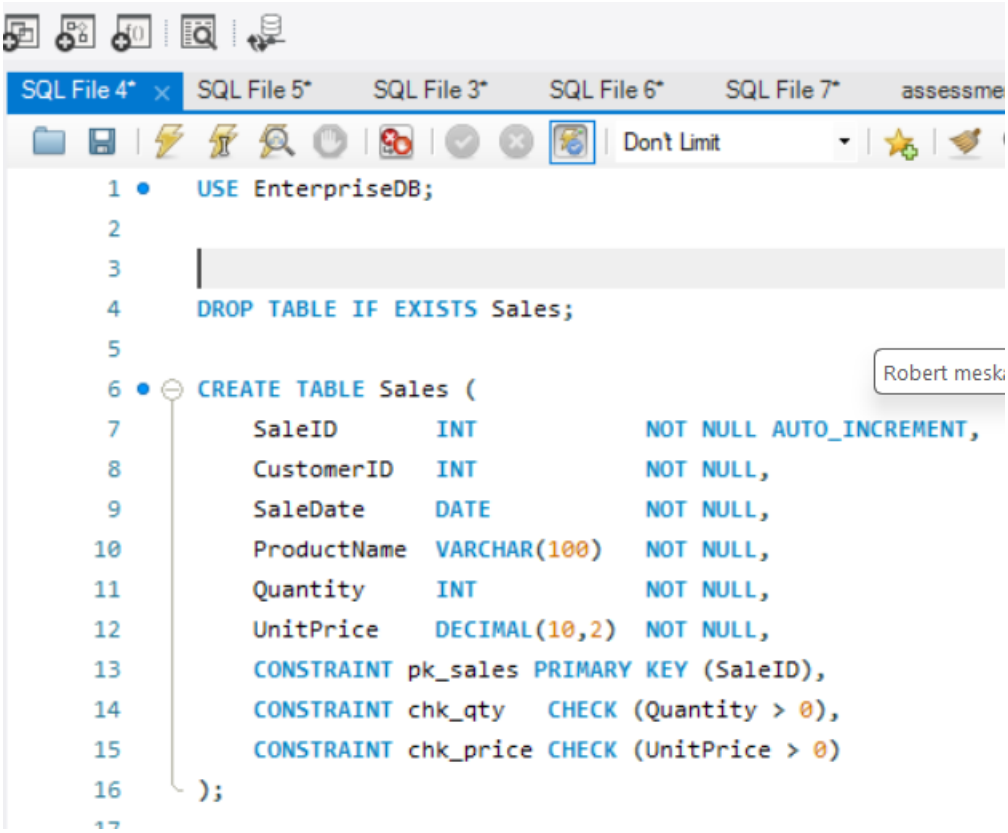
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

DepartmentName	EmployeeID	EmployeeName	Salary	SalaryRank
Finance	3	Rohan Iyer	135000.00	1
Finance	13	Amit Joshi	90000.00	2
Finance	14	Pooja Menon	78000.00	3
HR	2	Kaviya	140000.00	1
HR	10	Priya	70000.00	2
HR	11	Sanjay	65000.00	3
IT	1	Twinkle	150000.00	1
IT	5	Kimkimi	95000.00	2
IT	6	Dindini	85000.00	3
IT	7	Mali	85000.00	3
Sales	4	Harsitha	130000.00	1
Sales	15	Imran Khan	80000.00	2
Sales	16	Lakshmi Bhat	68000.00	3

Result 8

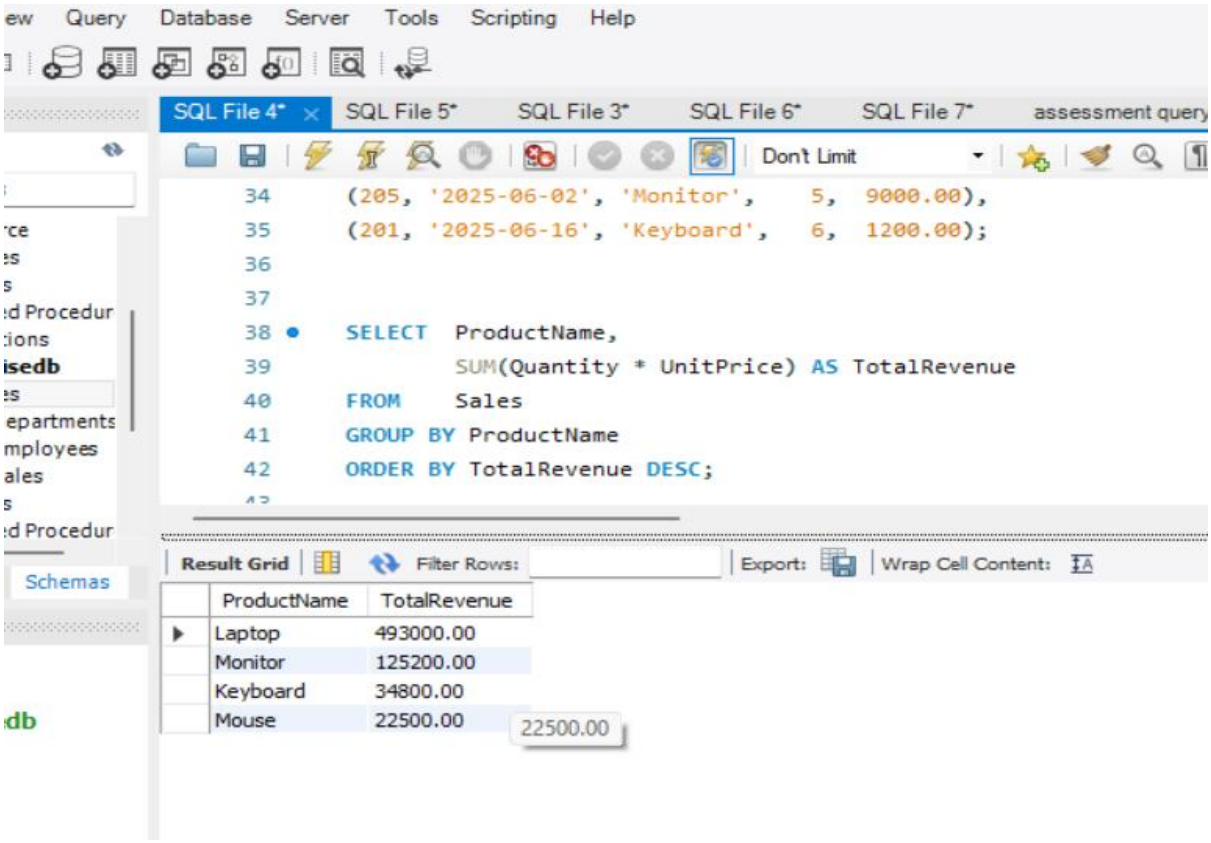
Question 2: Sales Analytics and Query Optimization (6 Marks)

Create a Sales table with the following columns: • SaleID • CustomerID • SaleDate • ProductName • Quantity • UnitPrice

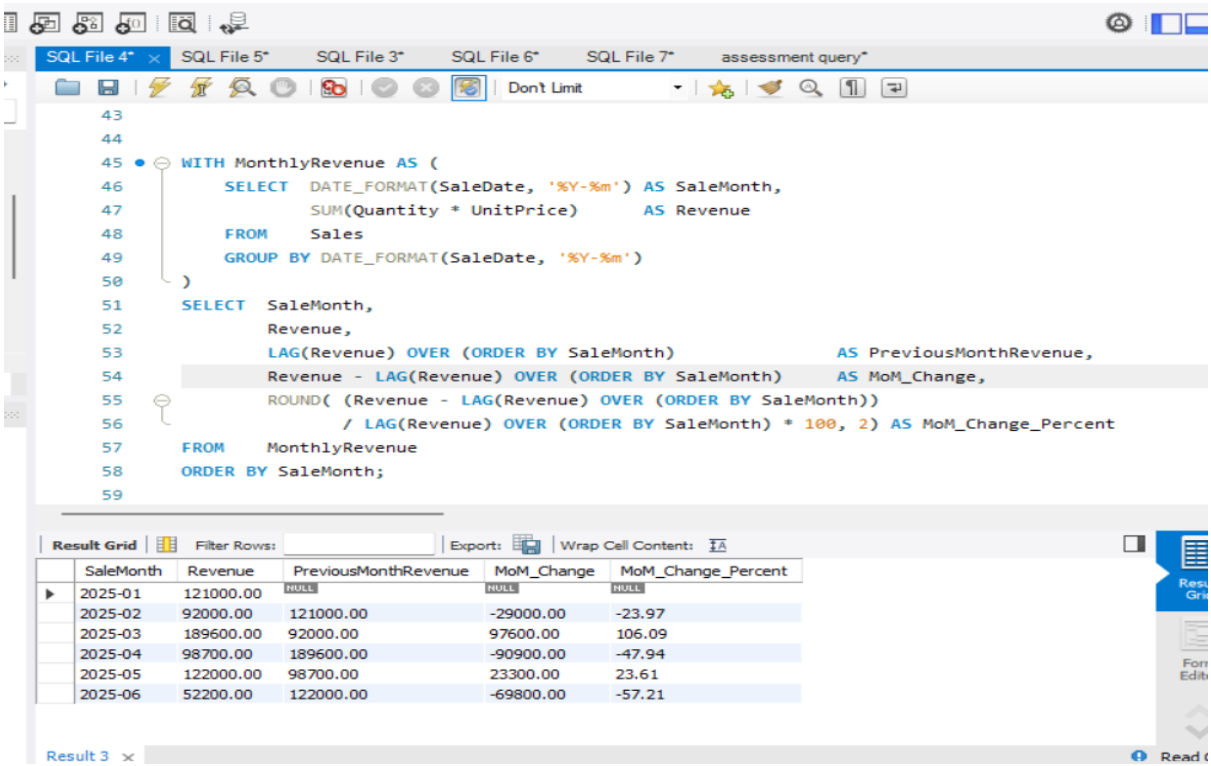


Write SQL queries to:

a. Calculate total revenue for each product. (2 marks)

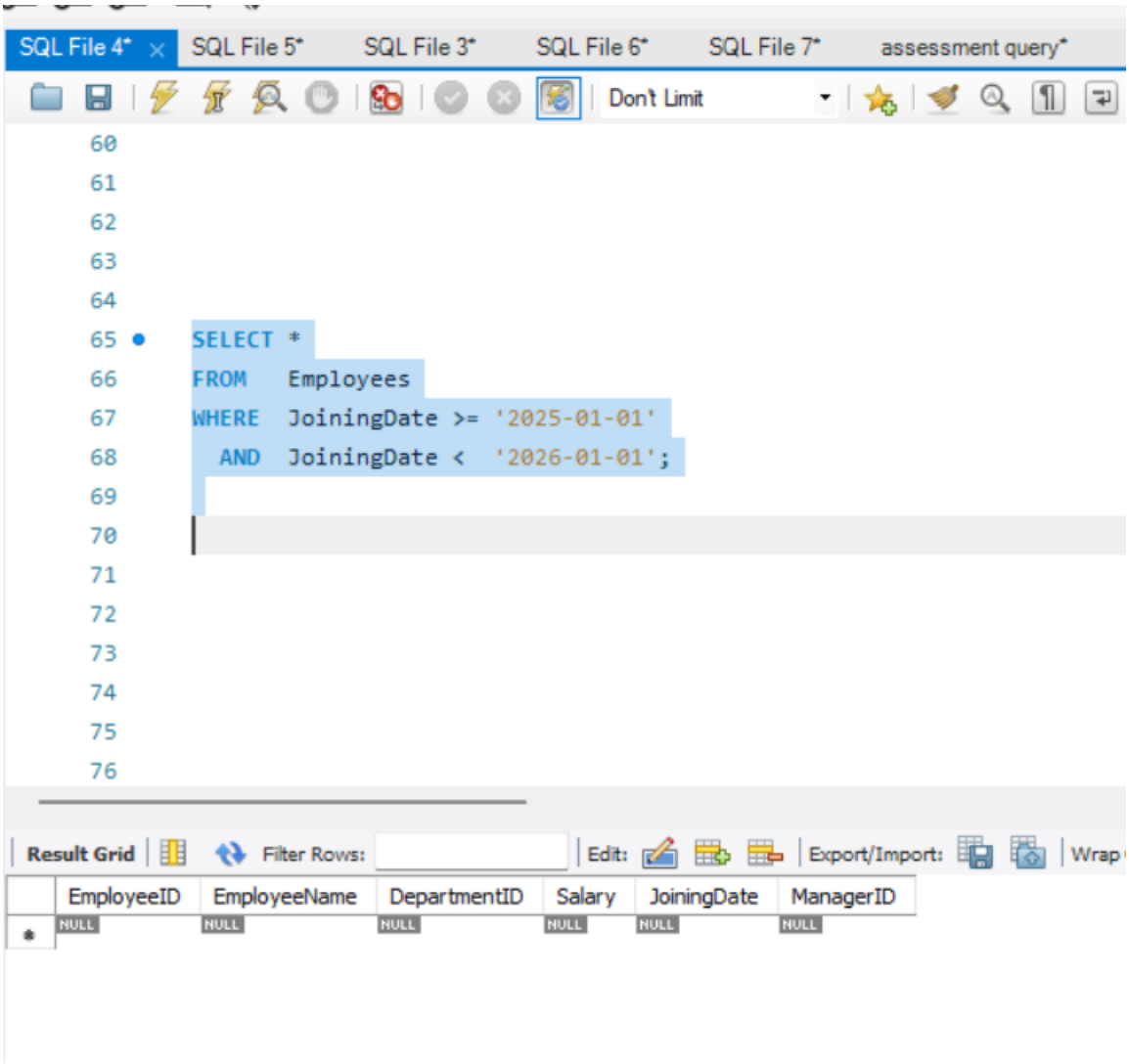


b. Calculate monthly revenue and month-over-month revenue change using a window function. (2 marks)



c. Rewrite the following query to improve the potential for index usage and explain why your version may perform better. (2 marks)

SELECT * FROM Employees WHERE YEAR(JoiningDate) = 2025; Assume JoiningDate may be indexed.



Question 3: SQL Troubleshooting (6 Marks)

A company maintains millions of employee and transaction records. Its reporting queries are running slowly.

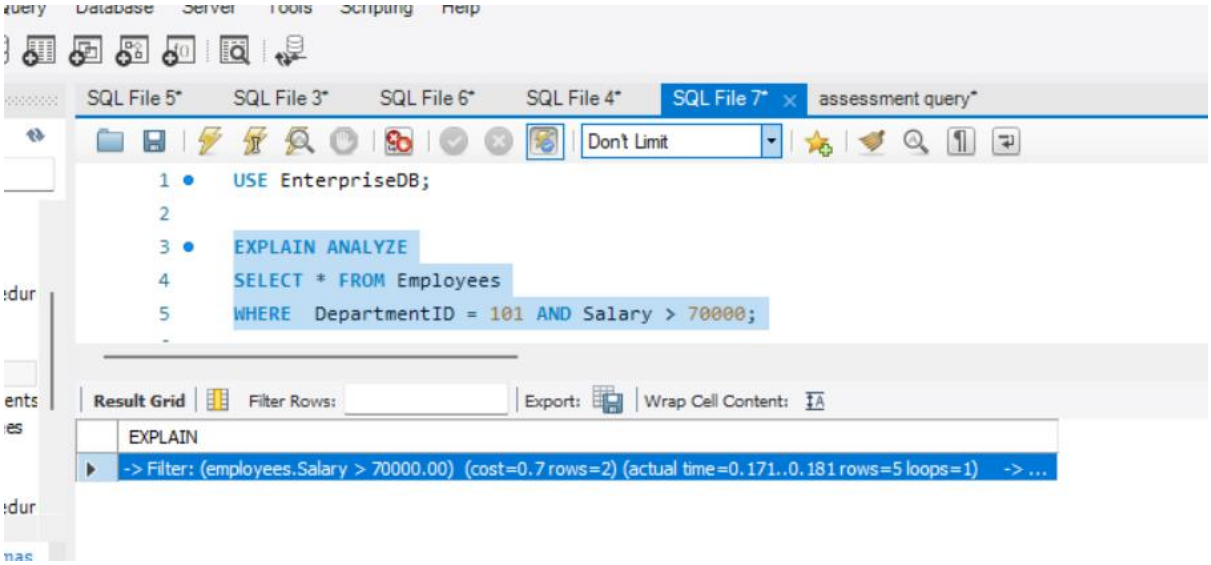
a. Explain how an execution plan helps identify expensive operations in a query. (2 marks)

An execution plan shows the steps the optimizer chose to run a query, so you can see where time and resources go instead of guessing. Things to check in it:

- Access method: A full table scan (type = ALL) on millions of rows is a red flag. An index seek or range scan (ref, range) reads only the relevant rows.

- Rows examined vs. rows returned: If the plan examines millions of rows to return a few hundred, the filter isn't being served by an index.
- Index usage: The possible_keys and key columns show whether an index exists and was actually used. Non-sargable conditions like YEAR(col) = 2025 cause the index to be ignored.
- Costly operators: Using filesort, Using temporary, large nested-loop or hash joins, and sorts spilling to disk are typical hot spots.
- Estimate vs. actual: Large gaps between estimated and actual row counts suggest stale statistics (fix with ANALYZE TABLE).

The plan turns "this query is slow" into a specific cause, and you can compare plans before and after a fix to confirm it worked.

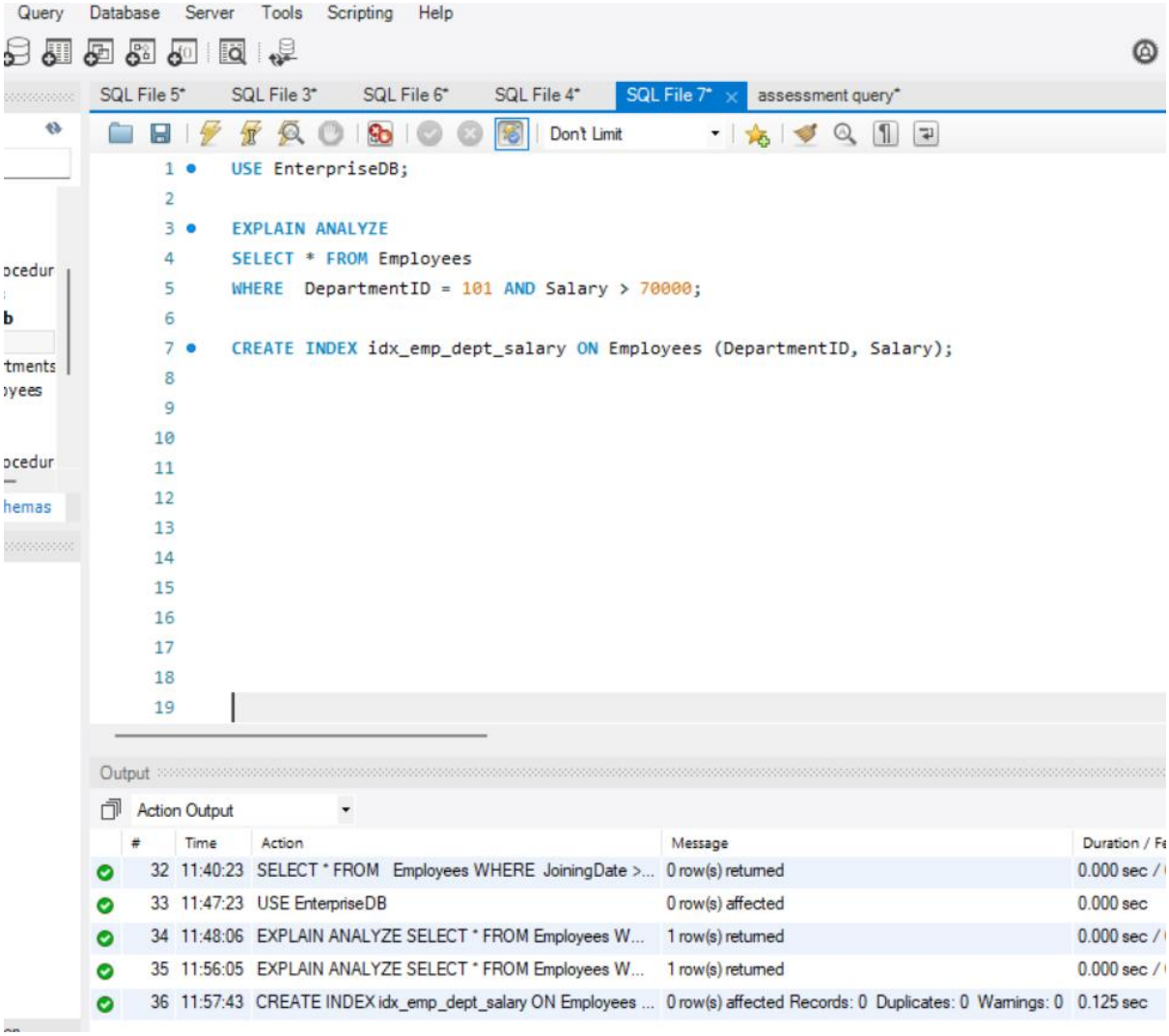


b. Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary. Explain your reasoning. (2 marks)

The recommended index is CREATE INDEX idx_emp_dept_salary ON Employees (DepartmentID, Salary);

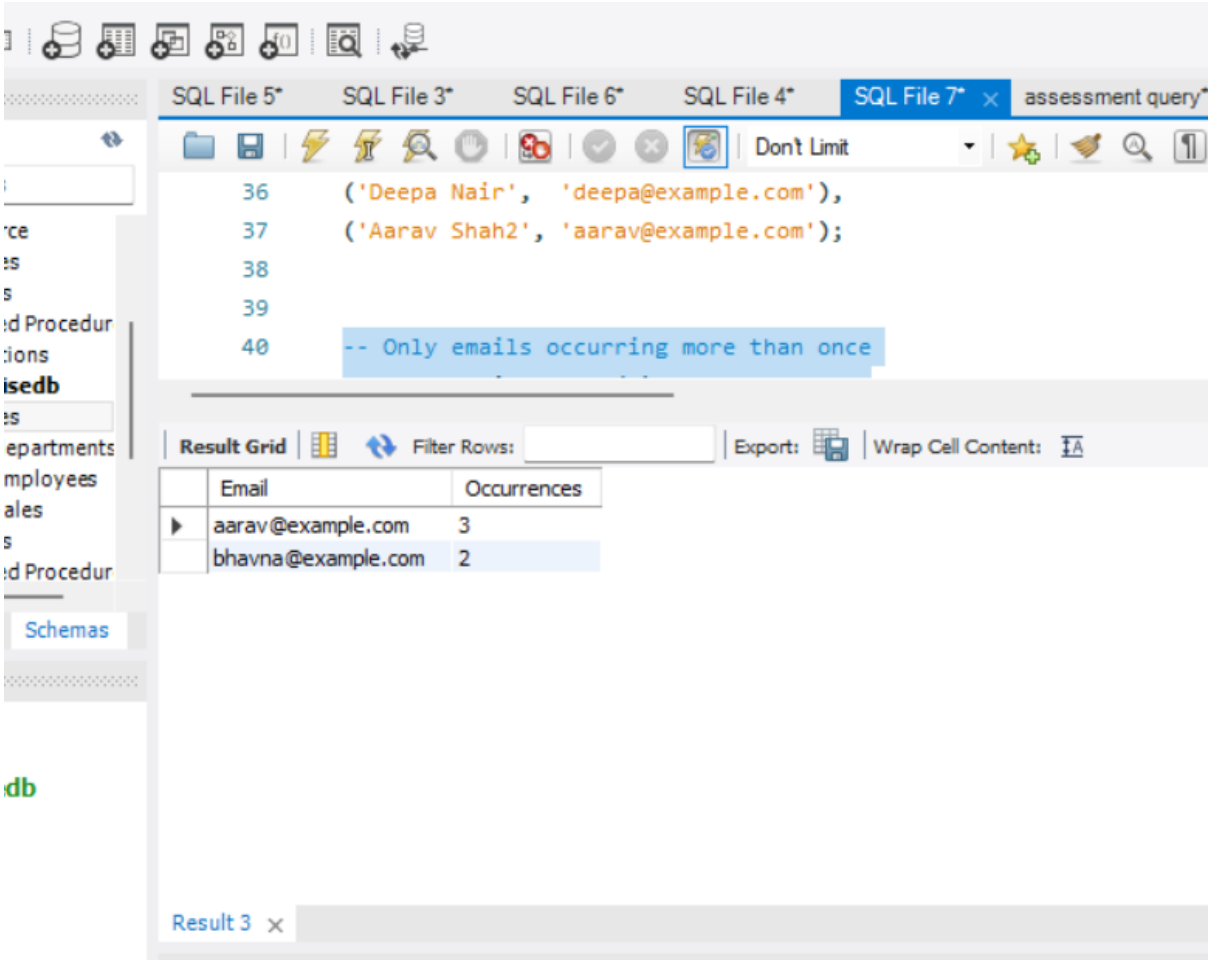
- Leftmost-prefix rule: The index serves queries filtering on DepartmentID alone and on DepartmentID plus Salary. It doesn't help queries filtering on Salary alone.
- Equality first, range second: For WHERE DepartmentID = 101 AND Salary > 70000, the engine seeks to department 101, then reads salaries in sorted order from 70000 upward. Reversing the column order would stop the index from narrowing by department.
- No extra sort: Within each department the entries are already ordered by Salary, so ORDER BY Salary and top-N queries avoid a filesort.
- Covering potential: A query selecting only indexed columns, such as DepartmentID and Salary, can be answered from the index without touching the table.

- Trade-off: Each index adds storage and slows INSERT, UPDATE and DELETE slightly, so create it only for proven query patterns. Two separate single-column indexes are usually less efficient than this one composite index



c. Write a query to identify duplicate customer email addresses and display only addresses occurring more than once. (2 marks)

GROUP BY Email collapses rows per address, and HAVING COUNT(*) > 1 keeps only addresses that occur more than once. WHERE can't be used here because it filters before grouping and can't reference aggregates. On the sample data it returns aarav@example.com (3) and bhavna@example.com (2). The second query normalizes case and spaces first, so Aarav@Example.com and aarav@example.com count as the same addres



SECTION B: LINUX || Maximum Marks: 15

Question 4: Log Analysis (5 Marks) Assume /var/log/application.log contains millions of log entries in the following format: 2026-10-28 10:15:23 ERROR payment-service Transaction failed Write Linux commands to:

a. Find all lines containing ERROR. (1 mark)

```
twinkle_meska@LAPTOP-1C20KDA8:~$ grep ' ERROR ' $LOG
2026-10-27 23:45:10 ERROR db-service Connection timeout
2026-10-28 09:15:42 ERROR payment-service Transaction failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:22:48 ERROR auth-service Invalid token
2026-10-28 10:45:37 ERROR db-service Query timeout
2026-10-28 11:02:19 ERROR payment-service Gateway unreachable
2026-10-28 11:35:12 ERROR order-service Order creation failed
2026-10-28 12:10:44 ERROR payment-service Transaction failed
2026-10-29 08:30:21 ERROR auth-service Invalid token
2026-10-29 09:12:33 ERROR db-service Deadlock detected
```

b. Display the 20 most recent error entries in the file. (1 mark)

```
twinkle_meska@LAPTOP-1C20KDA8:~$ grep ' ERROR ' $LOG | tail -n 20
2026-10-27 23:45:10 ERROR db-service Connection timeout
2026-10-28 09:15:42 ERROR payment-service Transaction failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:22:48 ERROR auth-service Invalid token
2026-10-28 10:45:37 ERROR db-service Query timeout
2026-10-28 11:02:19 ERROR payment-service Gateway unreachable
2026-10-28 11:35:12 ERROR order-service Order creation failed
2026-10-28 12:10:44 ERROR payment-service Transaction failed
2026-10-29 08:30:21 ERROR auth-service Invalid token
2026-10-29 09:12:33 ERROR db-service Deadlock detected
```

c. Count the total number of error entries. (1 mark)

```
twinkle_meska@LAPTOP-1C20KDA8:~$ grep -c ' ERROR ' $LOG
10
```

d. Extract error entries for a specified date into a separate file. (1 mark)

```
twinkle_meska@LAPTOP-1C20KDA8:~$ DATE="2026-10-28"
grep "^$DATE .* ERROR " $LOG > errors_$DATE.log
cat errors_$DATE.log
2026-10-28 09:15:42 ERROR payment-service Transaction failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:22:48 ERROR auth-service Invalid token
2026-10-28 10:45:37 ERROR db-service Query timeout
2026-10-28 11:02:19 ERROR payment-service Gateway unreachable
2026-10-28 11:35:12 ERROR order-service Order creation failed
2026-10-28 12:10:44 ERROR payment-service Transaction failed
```

e. Explain how you would calculate the number of errors occurring in each hour. (1 mark)

```
twinkle_meska@LAPTOP-1C20KDA8:~$ grep ' ERROR ' $LOG | awk '{print $1, substr($2,1,2)":00"
}' | sort | uniq -c
      1 2026-10-27 23:00
      1 2026-10-28 09:00
      3 2026-10-28 10:00
      2 2026-10-28 11:00
      1 2026-10-28 12:00
      1 2026-10-29 08:00
      1 2026-10-29 09:00
```

Question 5: Permissions and System Troubleshooting (5 Marks) A Linux server hosts an application that processes confidential financial reports. Perform the following tasks:

a. Create a group named finance and a report file, assuming you have the required privileges. (1 mark)

```
twinkle_meska@LAPTOP-1C20KDA8:~$ sudo groupadd finance
twinkle_meska@LAPTOP-1C20KDA8:~$ getent group finance
finance:x:1001:
twinkle_meska@LAPTOP-1C20KDA8:~$ sudo mkdir -p /srv/finance
twinkle_meska@LAPTOP-1C20KDA8:~$ sudo touch /srv/finance/financial_report.txt
twinkle_meska@LAPTOP-1C20KDA8:~$ echo "Confidential Q3 Financial Report" | sudo tee /srv/
finance/financial_report.txt
Confidential Q3 Financial Report
twinkle_meska@LAPTOP-1C20KDA8:~$ ls -l /srv/finance/financial_report.txt
-rw-r--r-- 1 root root 33 Oct 28 06:52 /srv/finance/financial_report.txt
twinkle_meska@LAPTOP-1C20KDA8:~$
```

b. Set permissions so the owner can read and write, the finance group can read, and other users have no access. (1 mark)

```
twinkle_meska@LAPTOP-1C20KDA8:~$ sudo chmod 640 /srv/finance/financial_report.txt
twinkle_meska@LAPTOP-1C20KDA8:~$ ls -l /srv/finance/financial_report.txt
-rw-r----- 1 root root 33 Oct 28 06:52 /srv/finance/financial_report.txt
twinkle_meska@LAPTOP-1C20KDA8:~$
```

c. Identify the processes consuming the most CPU. (1 mark)

```
twinkle_meska@LAPTOP-1C20KDA8:~$ ps aux --sort=-%cpu | head -n 6
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.3  0.4  24432 15444 ?        Ss   06:34   0:05 /sbin/init
twinkle+     444  0.1  0.3  22348 13204 ?        Ss   06:34   0:02 /usr/lib/systemd/syste
md --user
root          92  0.0  0.3  35320 12208 ?        Ss   06:34   0:00 /usr/lib/systemd/syste
md-udevd
root          52  0.0  0.4  50412 16812 ?        S<s  06:34   0:00 /usr/lib/systemd/syste
md-journald
root         260  0.0  0.8 123480 32696 ?        Ssl  06:34   0:00 /usr/bin/python3 /usr/
share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
twinkle_meska@LAPTOP-1C20KDA8:~$
```

d. Display memory usage and filesystem disk utilization. (1 mark)

```
twinkle_meska@LAPTOP-1C20KDA8:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.7Gi       443Mi       3.1Gi       3.5Mi       189Mi       3.2Gi
Swap:          1.0Gi          0B       1.0Gi
twinkle_meska@LAPTOP-1C20KDA8:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
none            1.9G     0  1.9G   0% /usr/lib/modules/6.18.33.2-microsoft-standard-
none            1.9G  4.0K  1.9G   1% /mnt/wsl
drivers         199G  177G   23G  89% /usr/lib/wsl/drivers
/dev/sdd       1007G  2.5G  954G   1% /
none            1.9G   40K  1.9G   1% /mnt/wslg
none            1.9G     0  1.9G   0% /usr/lib/wsl/lib
rootfs          1.9G  2.8M  1.9G   1% /init
none            1.9G  520K  1.9G   1% /run
none            1.9G     0  1.9G   0% /run/lock
none            1.9G     0  1.9G   0% /run/shm
none            1.9G   80K  1.9G   1% /mnt/wslg/versions.txt
none            1.9G   80K  1.9G   1% /mnt/wslg/doc
C:\             199G  177G   23G  89% /mnt/c
D:\             251G   25G  227G  10% /mnt/d
E:\             251G  2.3G  249G   1% /mnt/e
F:\             251G  2.0G  250G   1% /mnt/f
tmpfs           1.9G     0  1.9G   0% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
none            1.0M     0  1.0M   0% /run/credentials/getty@tty1.service
tmpfs           376M   12K  376M   1% /run/user/1000
twinkle_meska@LAPTOP-1C20KDA8:~$
```

e. Find files larger than 100 MB under a specified application directory. (1 mark)

```
twinkle_meska@LAPTOP-1C20KDA8:~$ sudo mkdir -p /opt/myapp
twinkle_meska@LAPTOP-1C20KDA8:~$ sudo fallocate -l 150M /opt/myapp/large_data.bin
twinkle_meska@LAPTOP-1C20KDA8:~$ sudo fallocate -l 10M /opt/myapp/small_data.bin
twinkle_meska@LAPTOP-1C20KDA8:~$ sudo find /opt/myapp -type f -size +100M
/opt/myapp/large_data.bin
twinkle_meska@LAPTOP-1C20KDA8:~$ sudo find /opt/myapp -type f -size +100M -exec ls -lh {}
 \;
-rw-r--r-- 1 root root 150M Oct 28 07:05 /opt/myapp/large_data.bin
twinkle_meska@LAPTOP-1C20KDA8:~$
```

Question 6: Bash Scripting (5 Marks) Write a Bash script named backup_check.sh that:

1. Accepts source and destination directories as command-line arguments.

```
twinkle_meska@LAPTOP-1C2(
winkle_meska@LAPTOP-1C20KDA8:~$ chmod +x backup_check.sh
winkle_meska@LAPTOP-1C20KDA8:~$ mkdir -p source_data
winkle_meska@LAPTOP-1C20KDA8:~$ echo "Customer transaction data" > source_data/sales.txt
winkle_meska@LAPTOP-1C20KDA8:~$ ./backup_check.sh source_data backup_folder
ackup successful: backup_folder/source_data_backup_20261028_201400.tar.gz
winkle_meska@LAPTOP-1C20KDA8:~$
```

2. Validates that both arguments are provided.

```
twinkle_meska@LAPTOP-1C20KDA8:~$ ./backup_check.sh
Error: Please provide source and destination directories.
Usage: ./backup_check.sh <source_directory> <destination_directory>
twinkle_meska@LAPTOP-1C20KDA8:~$
```

3. Checks whether the source directory exists.

```
twinkle_meska@LAPTOP-1C20KDA8:~$ ./backup_check.sh missing_folder backup_folder
Error: Source directory does not exist.
twinkle_meska@LAPTOP-1C20KDA8:~$
```

4. Creates the destination directory if it does not exist.

```
twinkle_meska@LAPTOP-1C20KDA8:~$ ls -ld backup_folder
drwxr-xr-x 2 twinkle_meska twinkle_meska 4096 Oct 28 20:14 backup_folder
twinkle_meska@LAPTOP-1C20KDA8:~$
```

5. Creates a timestamped compressed archive of the source directory.

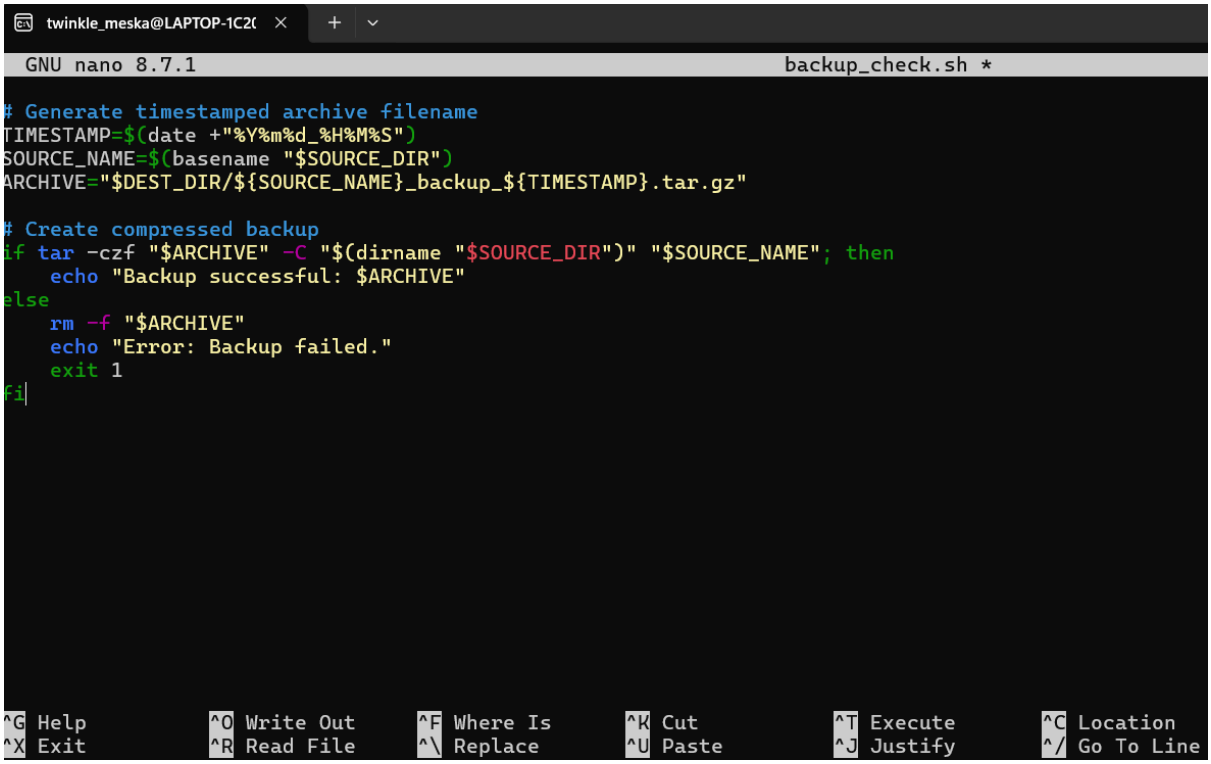
```
twinkle_meska@LAPTOP-1C20KDA8:~$ ls -lh backup_folder
total 8.0K
-rw-r--r-- 1 twinkle_meska twinkle_meska 190 Oct 28 20:10 source_data_backup_20261028_201052.tar.gz
-rw-r--r-- 1 twinkle_meska twinkle_meska 194 Oct 28 20:14 source_data_backup_20261028_201400.tar.gz
twinkle_meska@LAPTOP-1C20KDA8:~$
```

6. Displays a success message if the backup succeeds.

```
twinkle_meska@LAPTOP-1C20KDA8:~$ ./backup_check.sh source_data backup_folder
Backup successful: backup_folder/source_data_backup_20261028_202455.tar.gz
twinkle_meska@LAPTOP-1C20KDA8:~$
```

7. Displays an appropriate error message if the backup fails

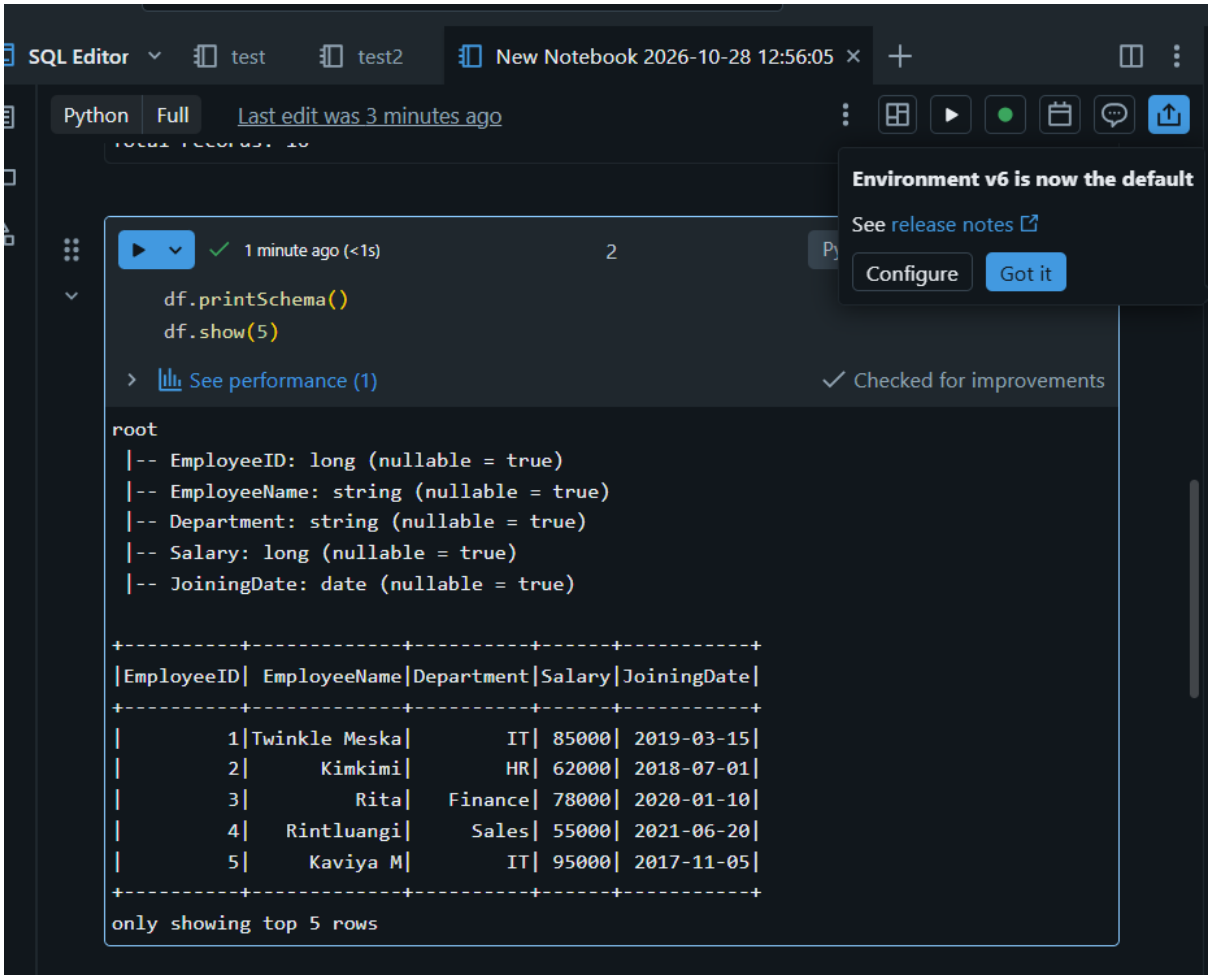
```
twinkle_meska@LAPTOP-1C20KDA8:~$ ./backup_check.sh missing_folder backup_folder
Error: Source directory does not exist.
twinkle_meska@LAPTOP-1C20KDA8:~$
```



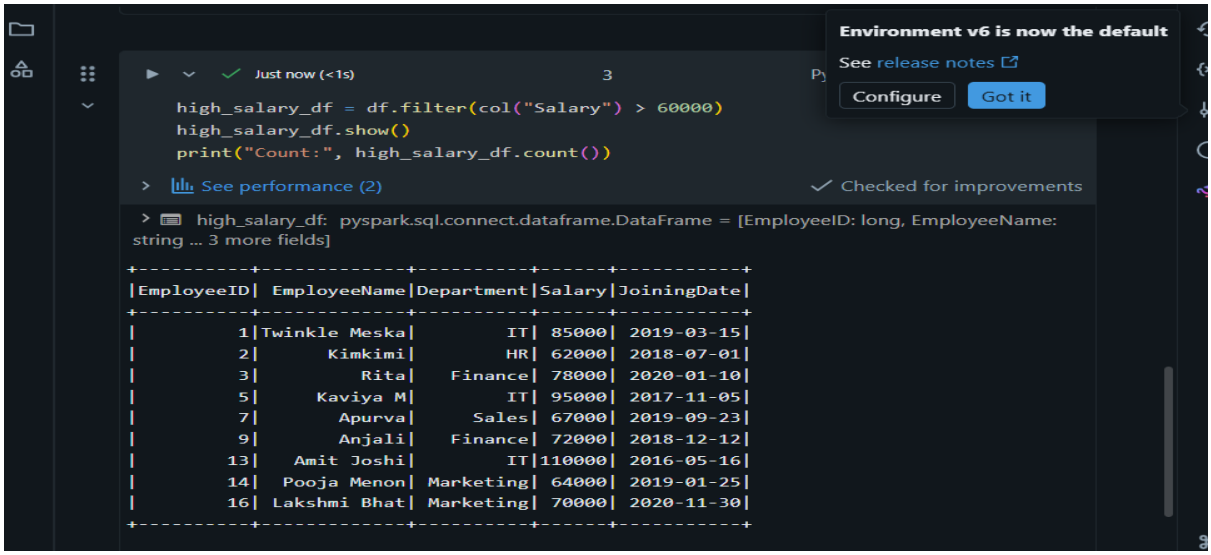
SECTION C: PYSPARK DATA ENGINEERING || Maximum Marks: 20

Question 7: PySpark DataFrame Operations (6 Marks) Create a PySpark DataFrame containing the following columns: • EmployeeID • EmployeeName • Department • Salary • JoiningDate Insert at least 15 sample records. Write and execute a PySpark program to:

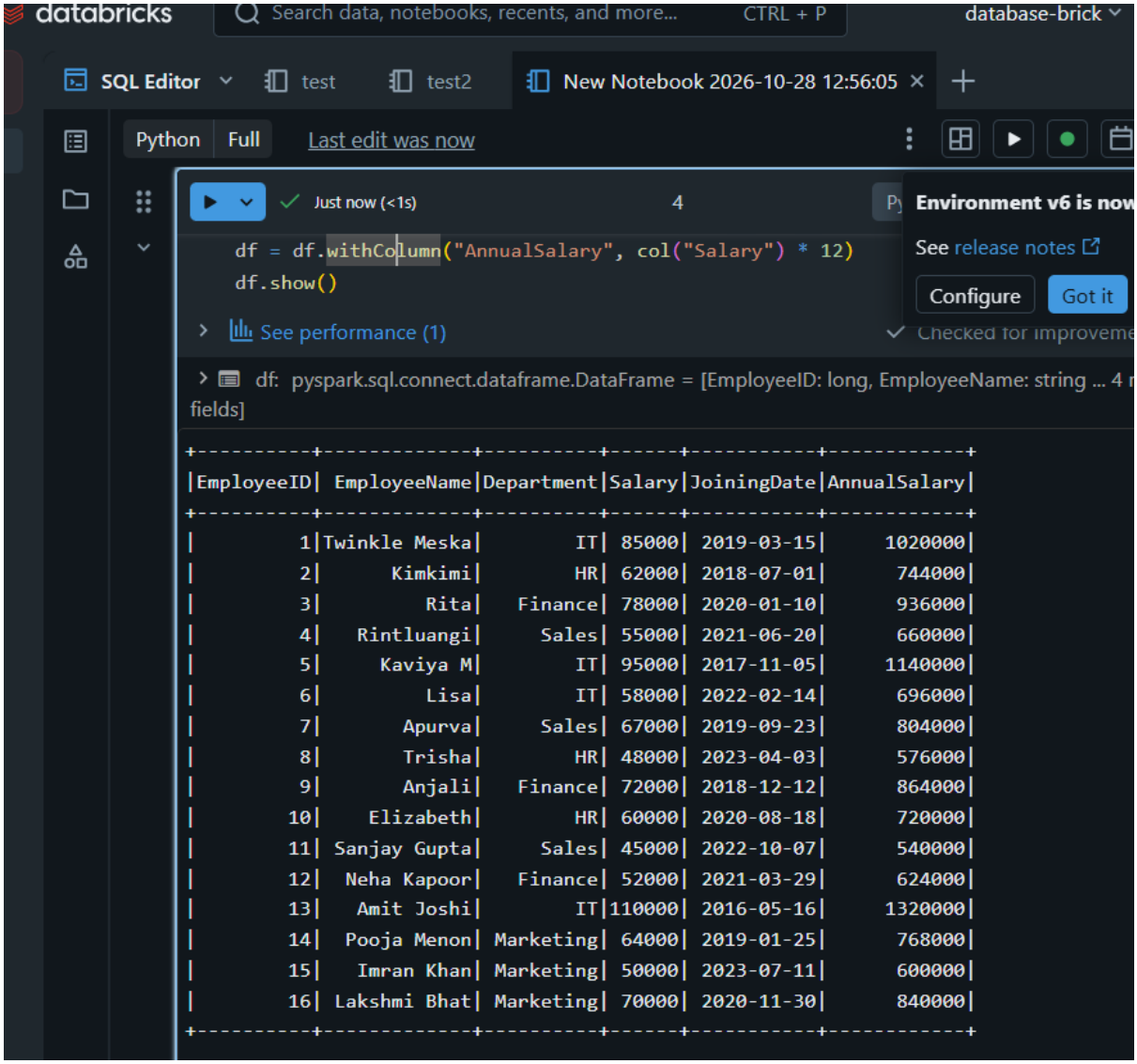
a. Display the schema and first five records. (1 mark)



b. Filter employees whose monthly salary is greater than ₹60,000. (1 mark)

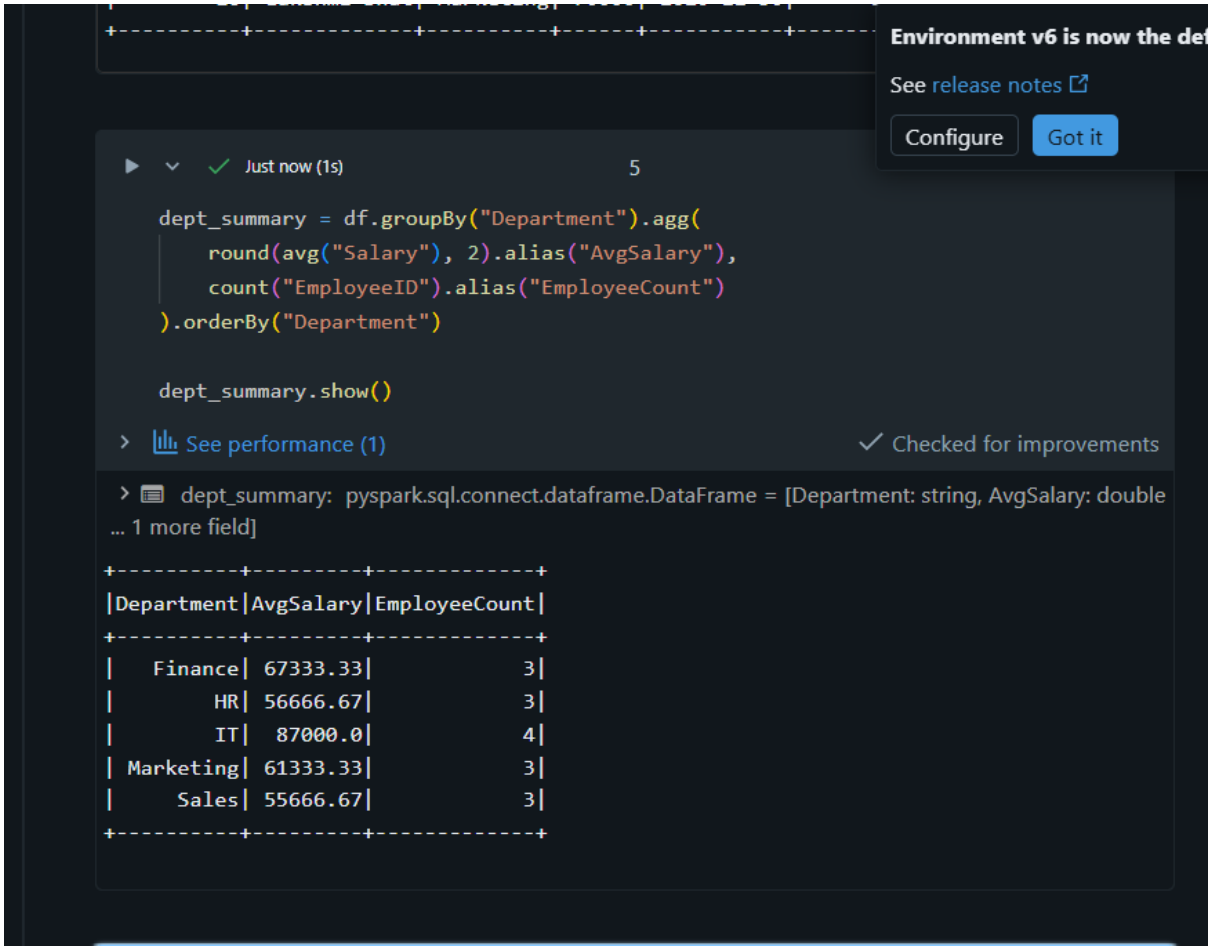


c. Add an AnnualSalary column calculated from the monthly salary. (1 mark)

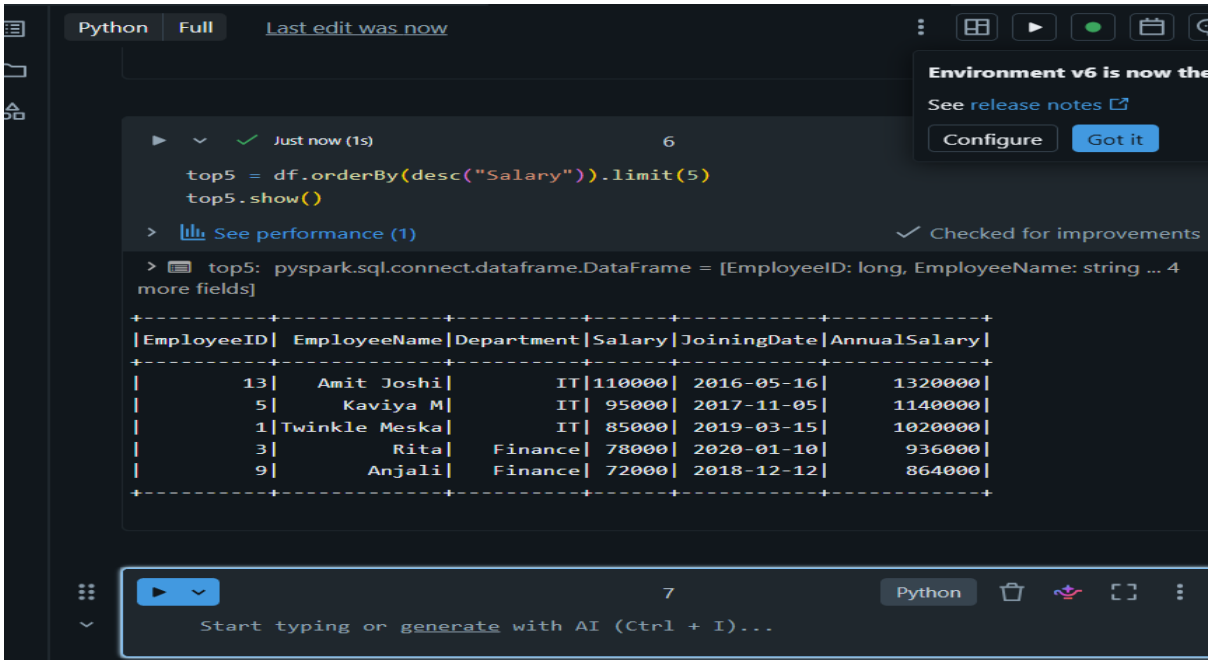


EmployeeID	EmployeeName	Department	Salary	JoiningDate	AnnualSalary
1	Twinkle Meska	IT	85000	2019-03-15	1020000
2	Kimkimi	HR	62000	2018-07-01	744000
3	Rita	Finance	78000	2020-01-10	936000
4	Rintluangi	Sales	55000	2021-06-20	660000
5	Kaviya M	IT	95000	2017-11-05	1140000
6	Lisa	IT	58000	2022-02-14	696000
7	Apurva	Sales	67000	2019-09-23	804000
8	Trisha	HR	48000	2023-04-03	576000
9	Anjali	Finance	72000	2018-12-12	864000
10	Elizabeth	HR	60000	2020-08-18	720000
11	Sanjay Gupta	Sales	45000	2022-10-07	540000
12	Neha Kapoor	Finance	52000	2021-03-29	624000
13	Amit Joshi	IT	110000	2016-05-16	1320000
14	Pooja Menon	Marketing	64000	2019-01-25	768000
15	Imran Khan	Marketing	50000	2023-07-11	600000
16	Lakshmi Bhat	Marketing	70000	2020-11-30	840000

d. Group employees by department and calculate the average salary and employee count. (2 marks)



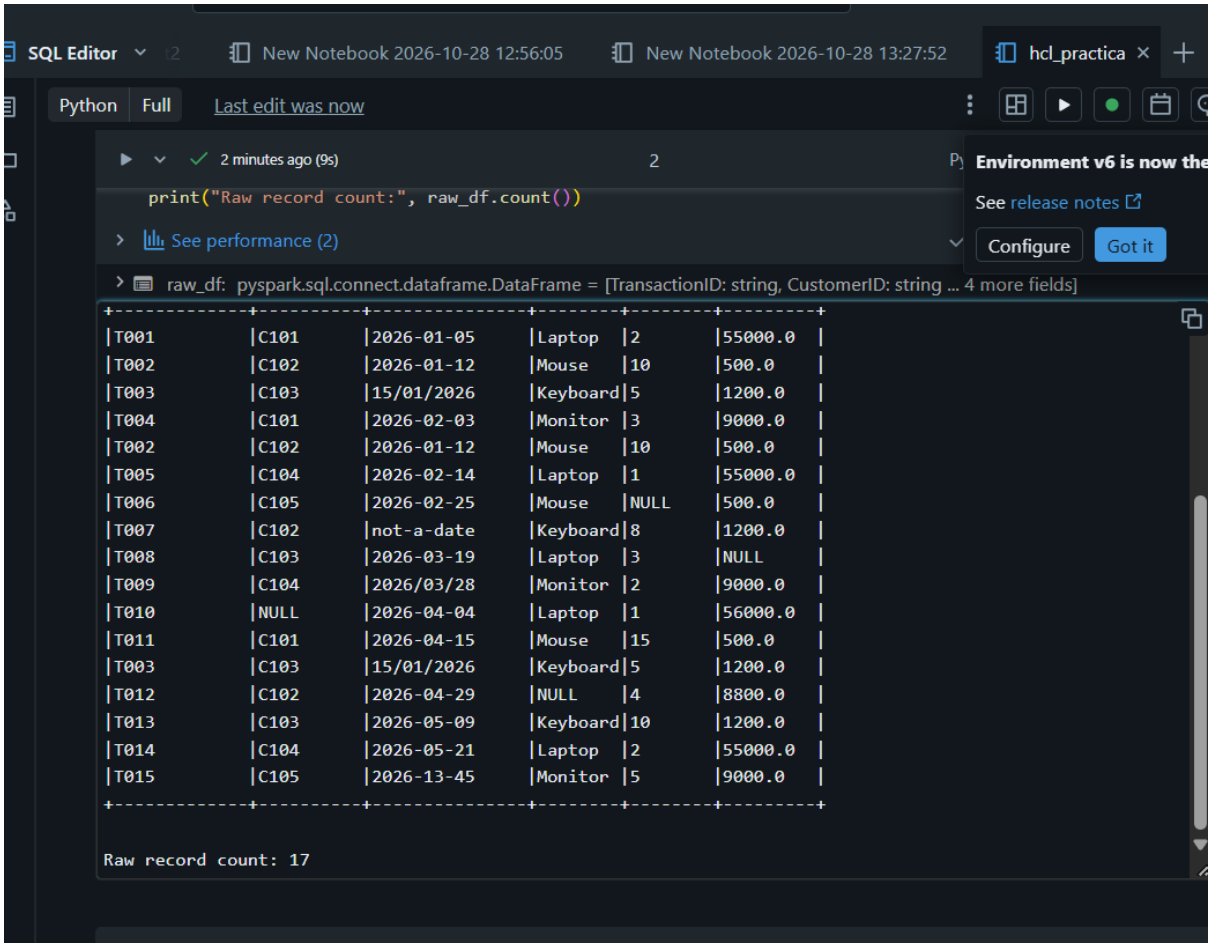
e. Display the five highest-paid employees. (1 mark) Expected Output: Submit the complete PySpark program and execution results.



Question 8: Data Cleaning and Aggregation (7 Marks)

A company receives a CSV file named transactions.csv containing: • TransactionID • CustomerID • TransactionDate • Product • Quantity • UnitPrice The file contains duplicate records, missing values, and incorrectly formatted dates. Write a PySpark program to:

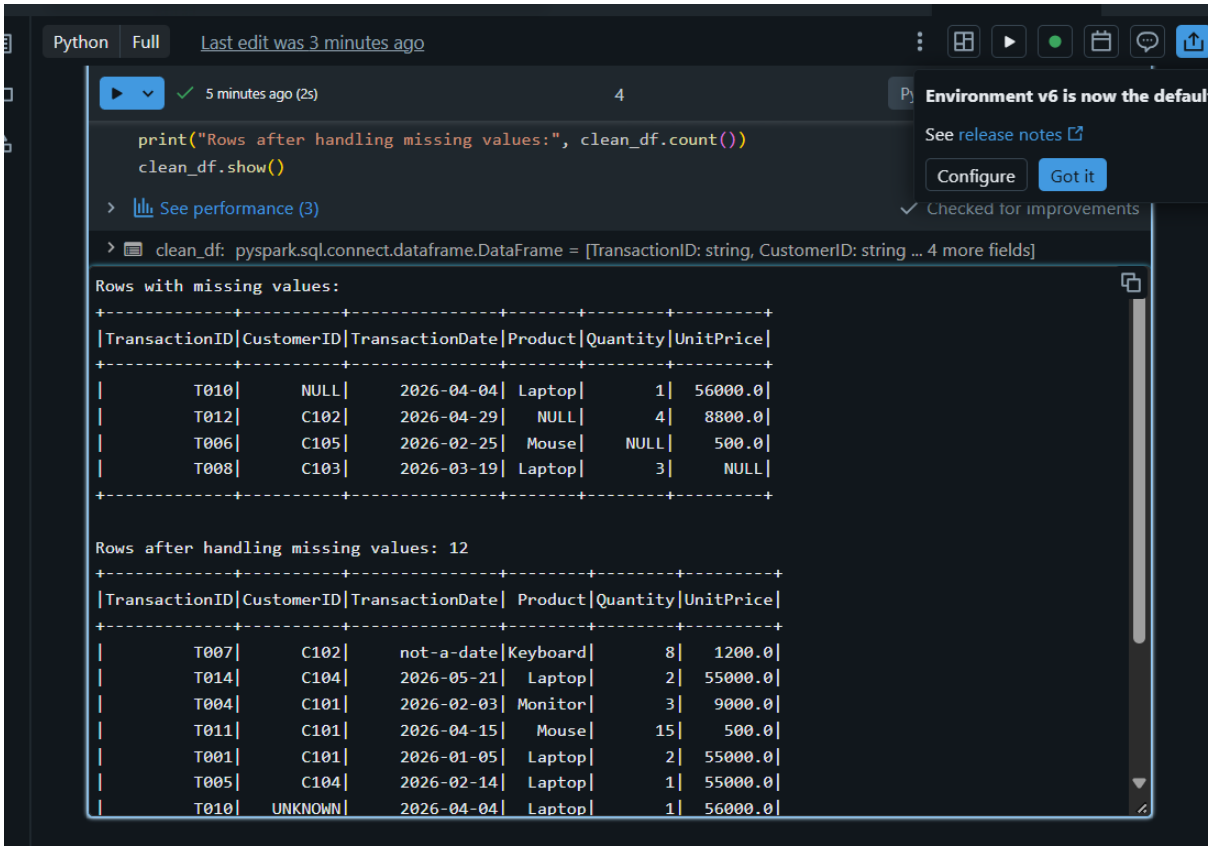
a. Read the CSV file using an appropriate schema and header configuration. (1 mark)



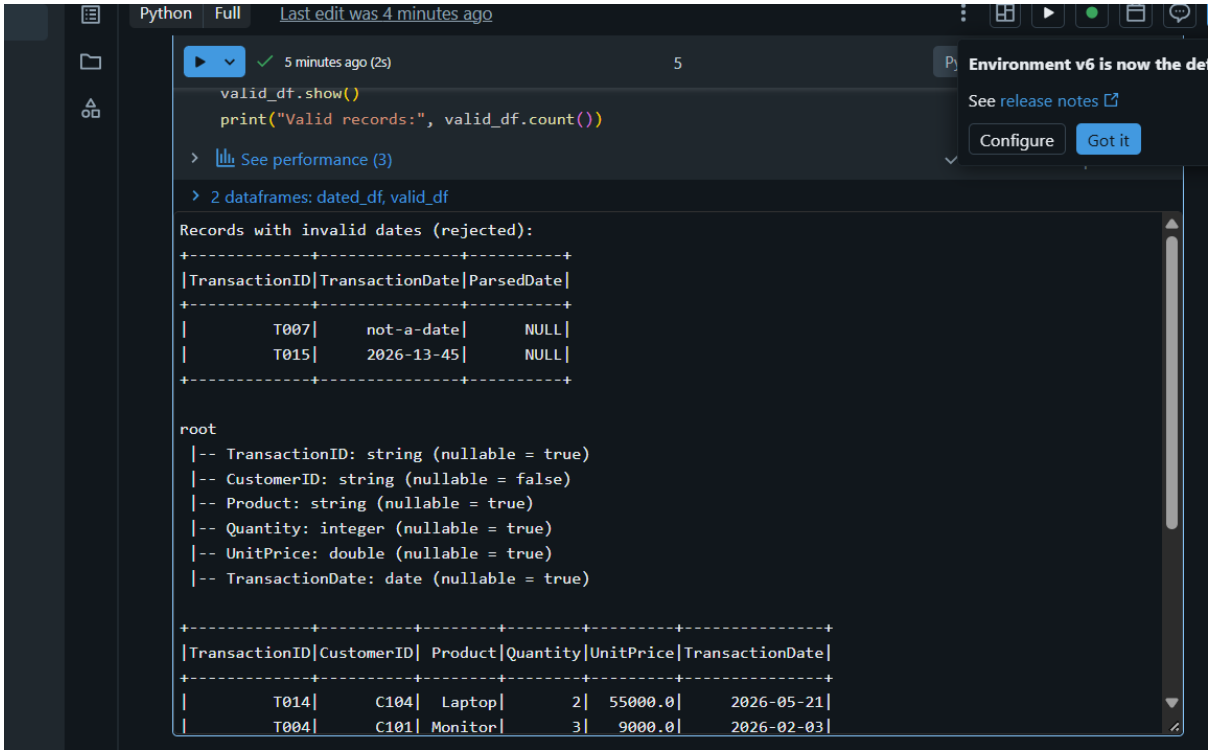
b. Identify and remove duplicate transactions using a suitable business key. (1 mark)



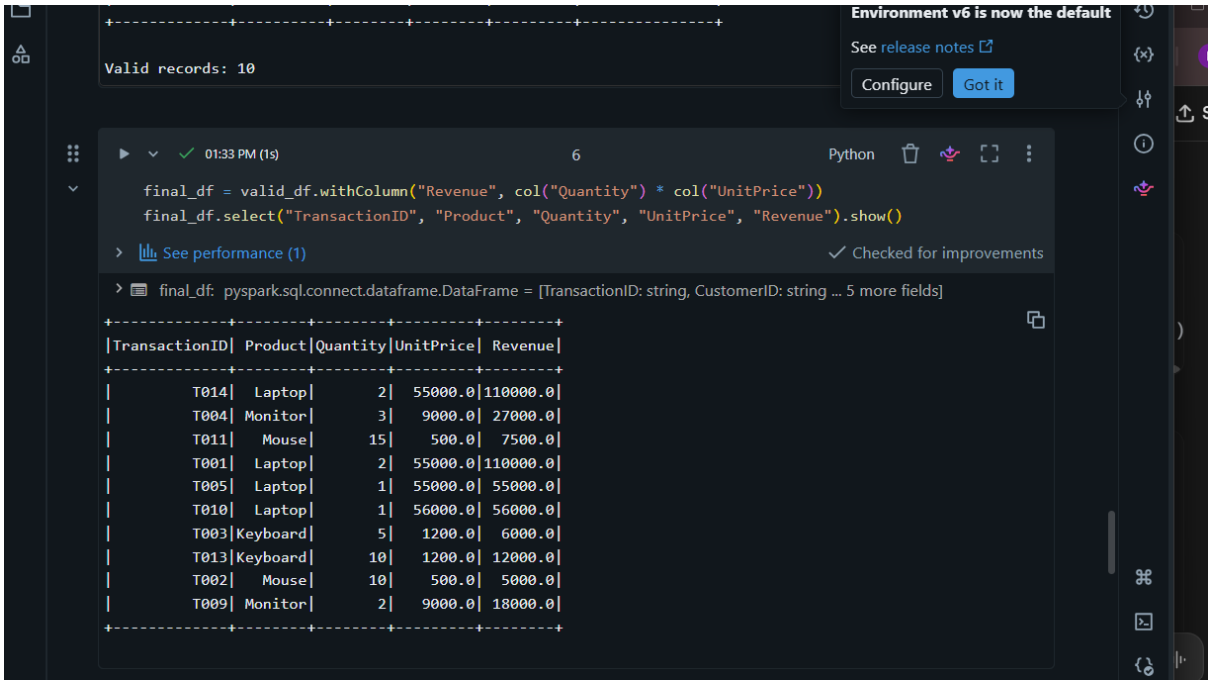
c. Handle missing values in required fields and explain your chosen approach. (2 marks)



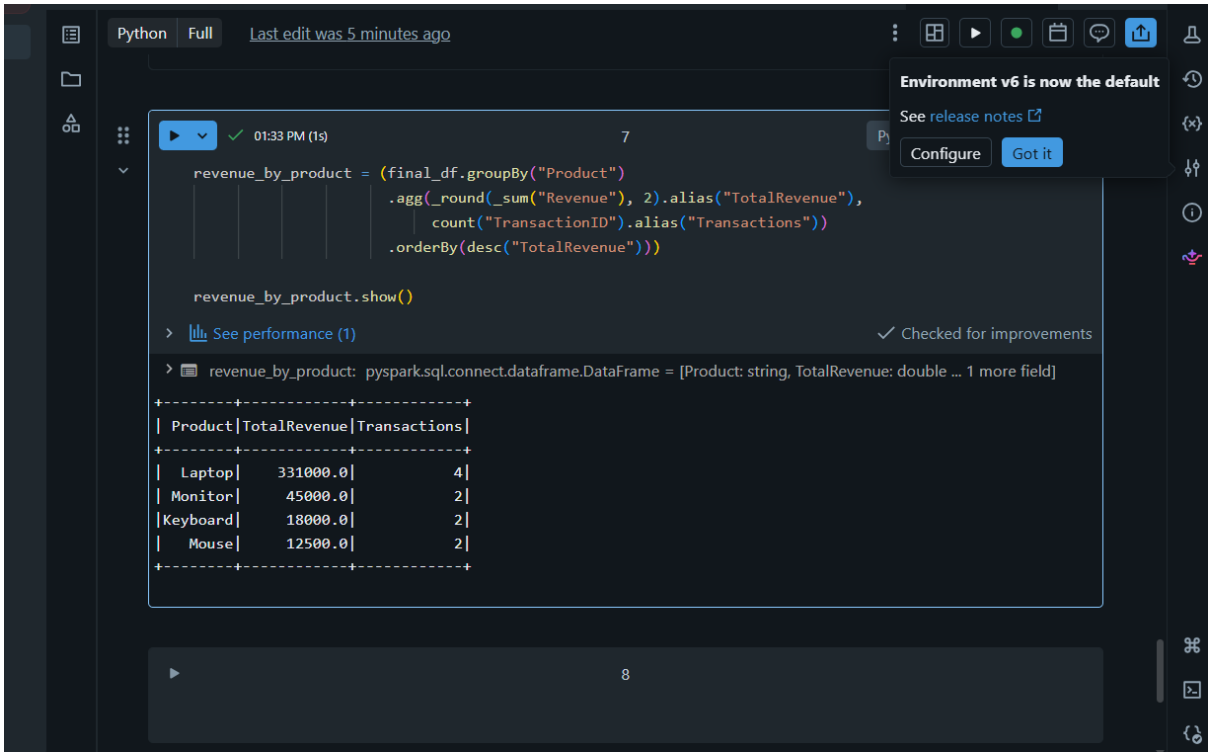
d. Convert TransactionDate into a date type and handle invalid dates. (1 mark)



e. Calculate revenue using Quantity × UnitPrice. (1 mark)



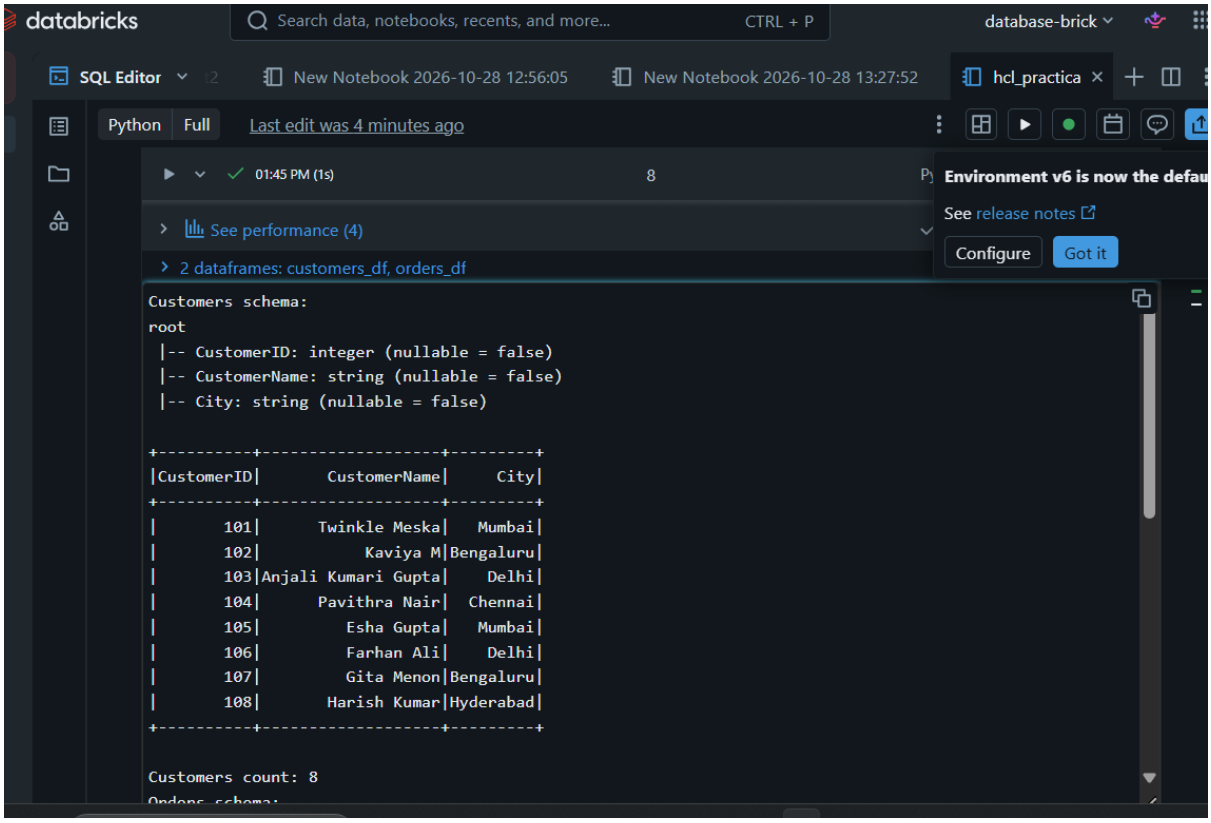
f. Calculate total revenue by product and display the results in descending order. (1 mark)



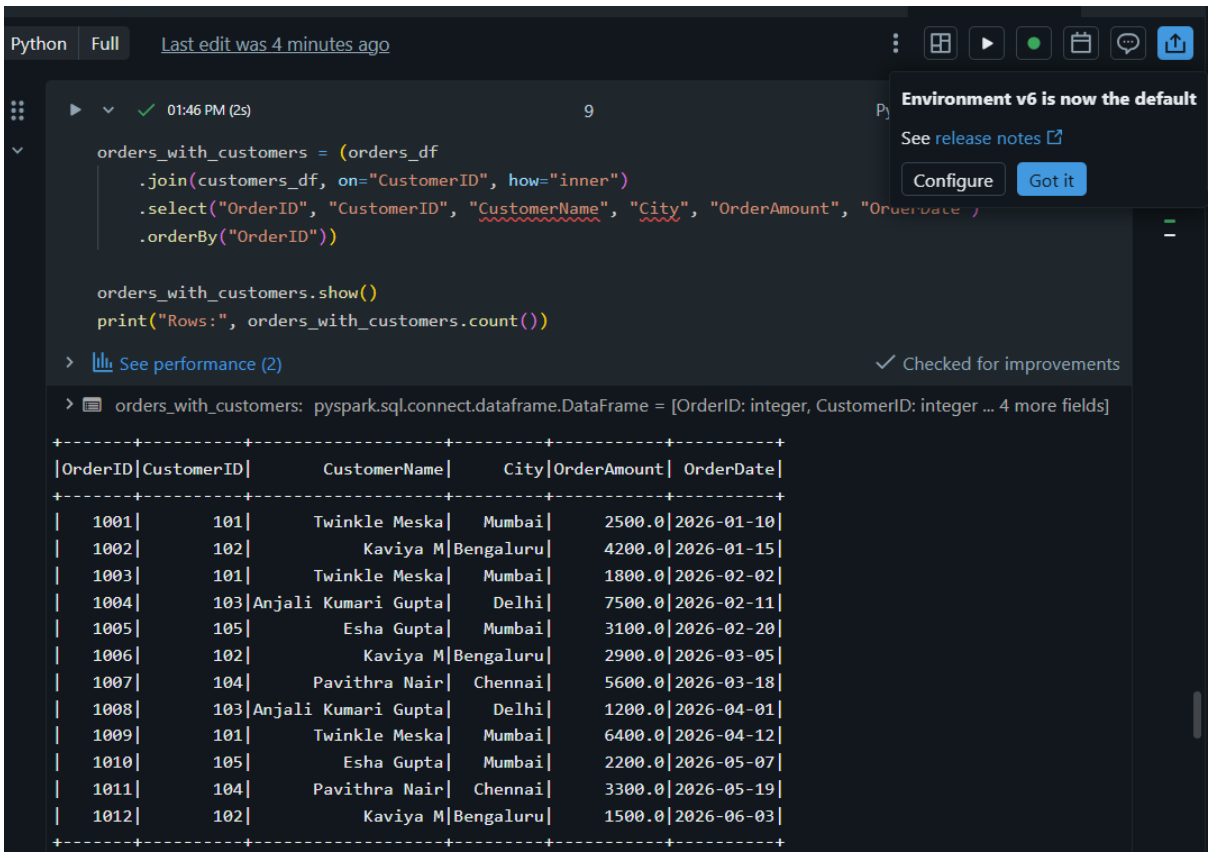
Question 9: PySpark Joins and Performance (7 Marks)

A company maintains two datasets. Customers • CustomerID • CustomerName • City Orders • OrderID • CustomerID • OrderAmount • OrderDate Write a PySpark program to:

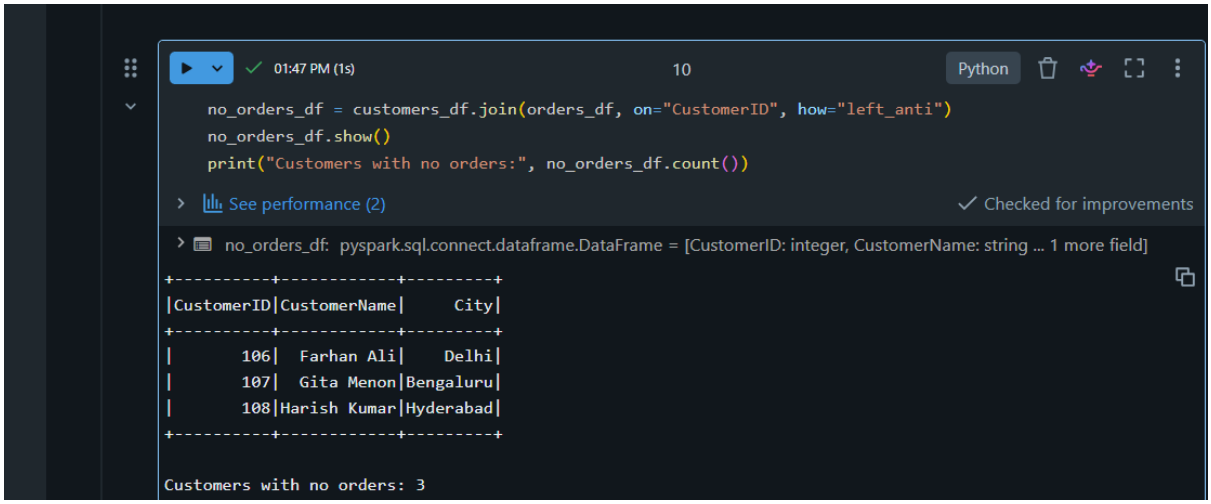
a. Load both datasets into DataFrames. (1 mark)



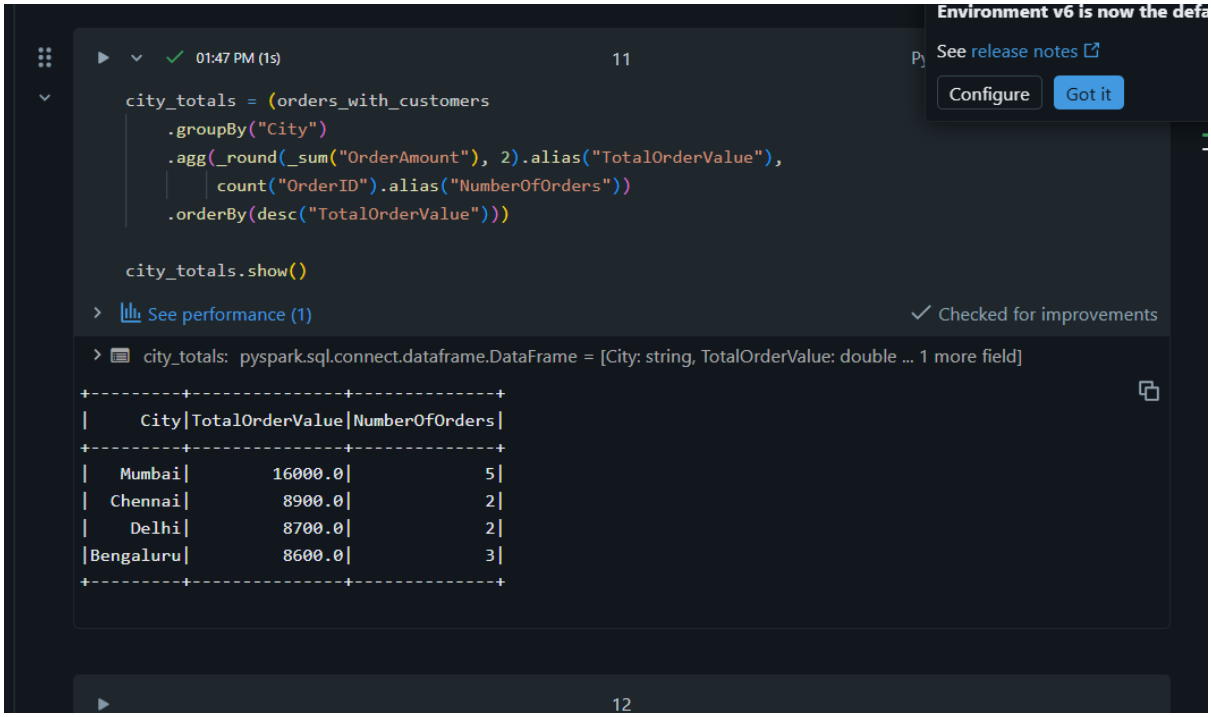
b. Join the datasets to display orders with customer details. (2 marks)



c. Identify customers who have never placed an order. (1 mark)



d. Calculate the total order value by city. (1 mark)



e. Explain when a broadcast join is useful and what conditions make it appropriate. (1 mark)

A broadcast join sends a full copy of the smaller DataFrame to every executor, so each executor joins its own partitions of the large DataFrame locally. This avoids shuffling the large table across the network, which is usually the most expensive part of a normal join (sort-merge join).

It is appropriate when:

- One side is small enough to fit in memory on the driver and on every executor. The default spark.sql.autoBroadcastJoinThreshold is about 10 MB.
- The other side is large, such as a big fact table joined with a small dimension or lookup table. In this example, a small Customers table joined with a large Orders table is a typical case.
- The join is an equi-join on a key (CustomerID), which Spark handles best as a broadcast hash join.

- The small table is not the preserved side of an outer join. For a left outer join, only the right table can be broadcast.

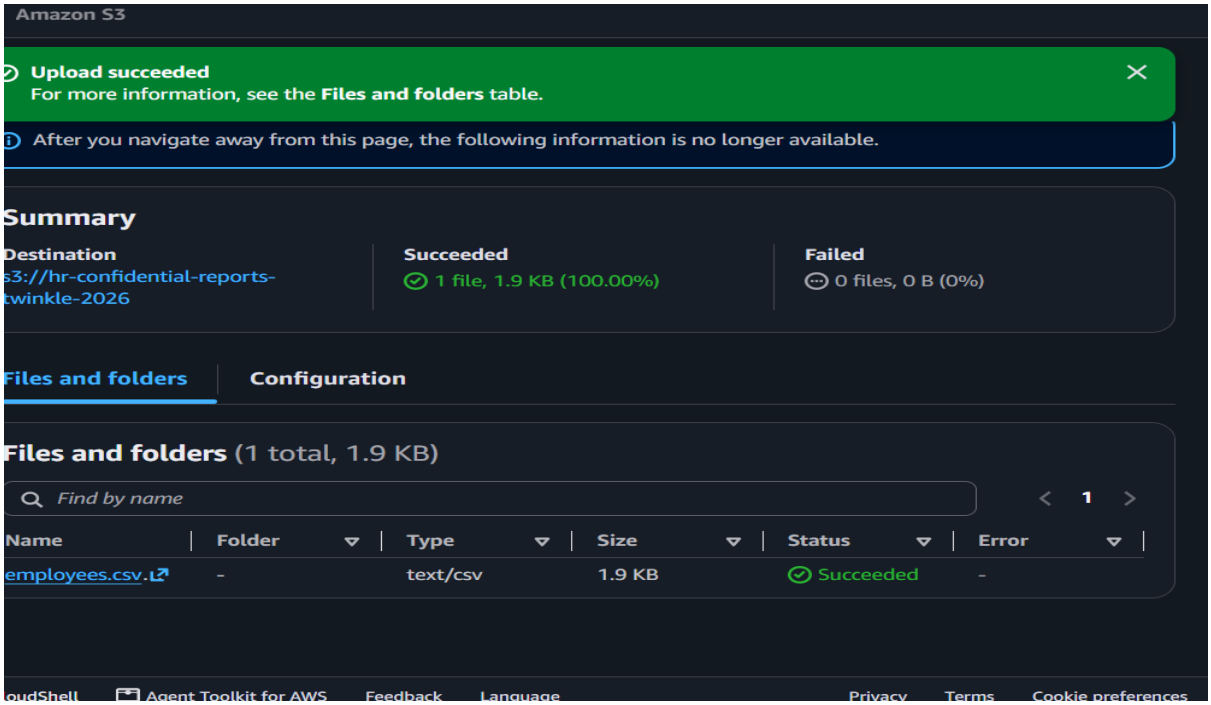
f. Explain the difference between transformations and actions, giving one example of each. (1 mark)

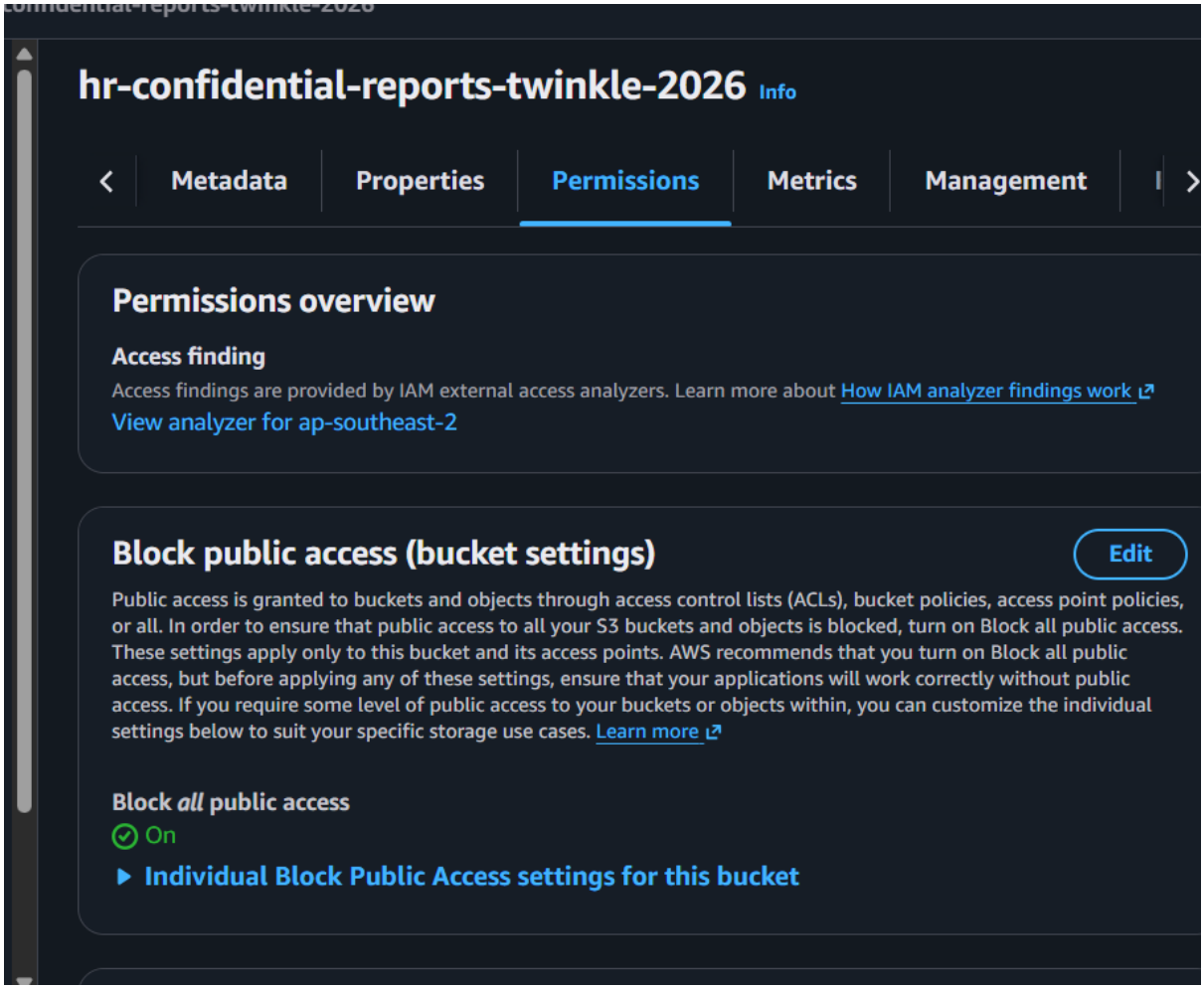
Transformation	Action
Defines a new DataFrame from an existing one	Triggers actual computation and returns a result to the driver or writes output
Lazy: nothing runs yet, Spark only builds a plan	Eager: runs the whole plan built so far
A new DataFrame	A value, rows, or a side effect (such as saved files)
orders_df.filter(col("OrderAmount") > 3000)	orders_df.count()

SECTION D: AWS CLOUD SERVICES || Maximum Marks: 15

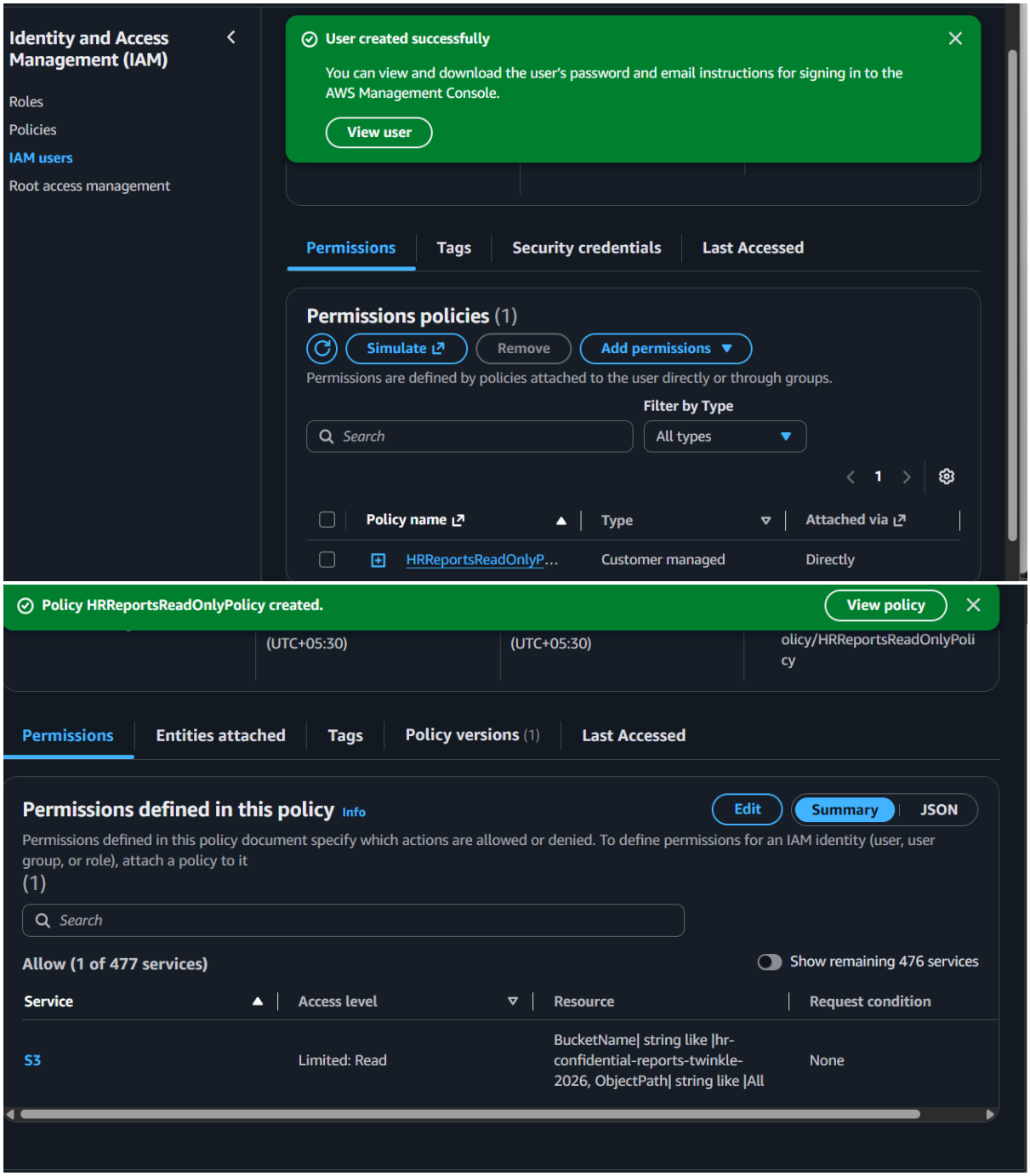
Question 10: S3 and IAM Security (5 Marks) A company stores confidential employee reports in Amazon S3. Only authorized HR employees should be able to read these reports. Perform or document the following:

a. Create an S3 bucket with Block Public Access enabled and upload a sample file. (1 mark)

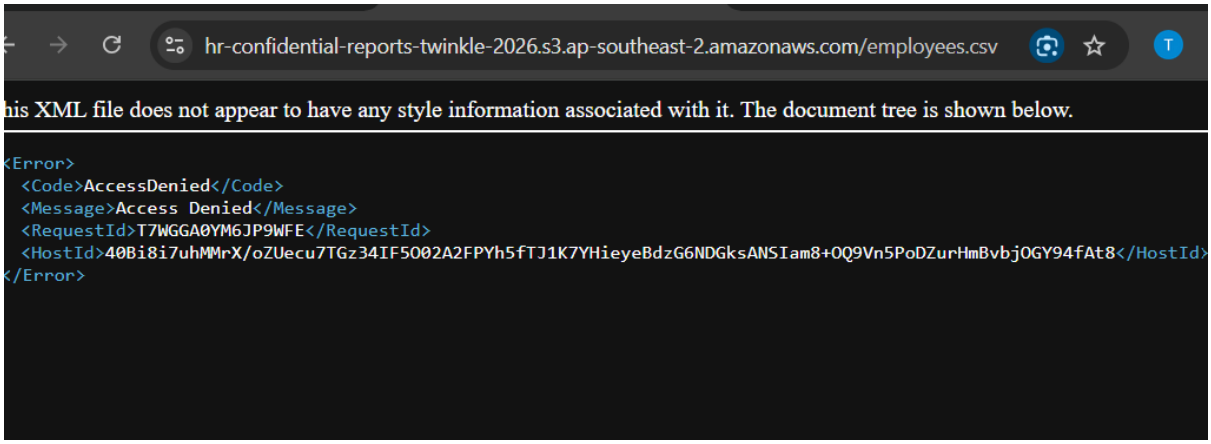




b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets. (2 marks)



c. Explain how you would verify that an unauthorized user cannot read the file. (1 mark)



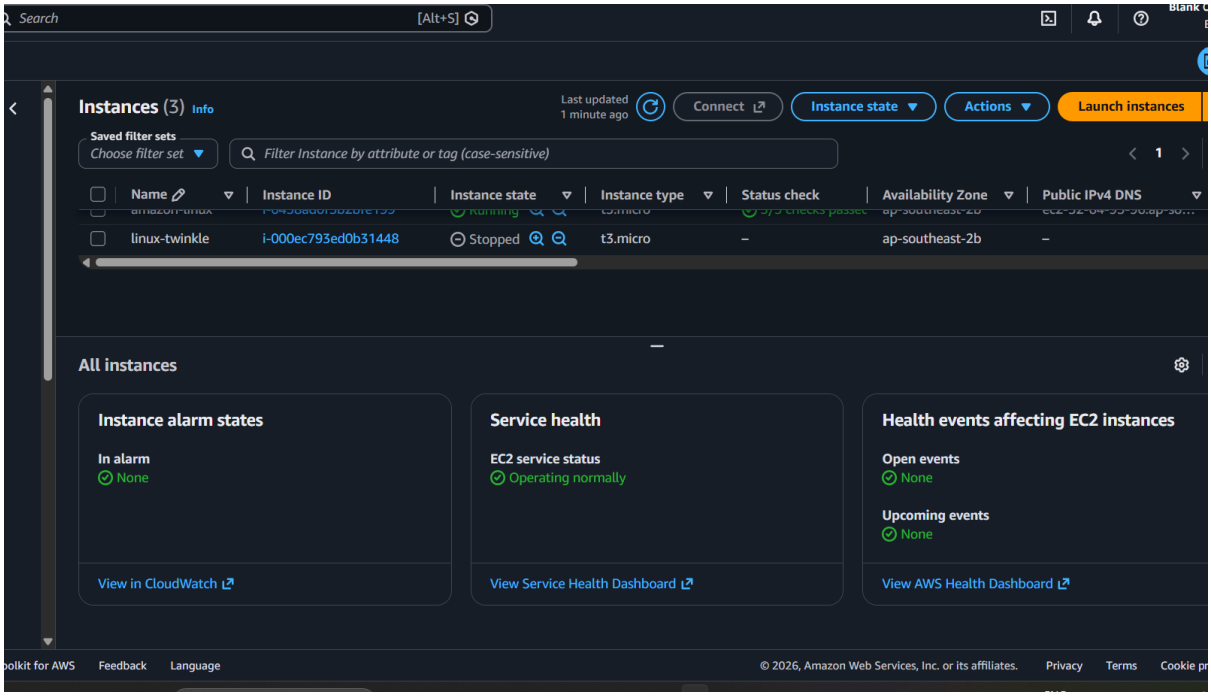
d. Explain how encryption at rest protects the stored data. (1 mark)

Amazon S3 encryption at rest protects stored files by encrypting their contents while they are saved on S3. S3 supports server-side encryption, including SSE-S3 and SSE-KMS.

Even if someone obtains the underlying stored data, encryption helps prevent them from reading its contents without the required decryption capability. Encryption does not replace IAM permissions; both access control and encryption are important.

Question 11: EC2 Troubleshooting and CloudWatch (5 Marks) A web application hosted on EC2 becomes slow, and users report connectivity problems. Perform or document the following:

a. Check the EC2 instance state and status checks. (1 mark)



b. Explain how security groups and route tables can affect application connectivity. (1 mark)

Security groups = virtual firewall

They control inbound and outbound traffic for the EC2 instance.

- Web application: allow inbound HTTP port 80 or HTTPS port 443, as required.
- SSH: allow port 22 only from an authorized IP address.
- Incorrect rules can block users from connecting.

Route tables = traffic directions

They determine where network traffic goes.

- A public subnet generally needs a route such as 0.0.0.0/0 → Internet Gateway for IPv4 internet connectivity.
- A missing or incorrect route can prevent internet connectivity even if the security group allows traffic.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period. (2 marks)

Step 1: Open CloudWatch

Go to the CloudWatch Console in the same AWS Region as your EC2 instance.

Select Alarms → All alarms → Create alarm.

Step 2: Select the metric

1. Click Select metric.
2. Choose EC2 → Per-Instance Metrics.
3. Find your instance ID.
4. Select CPUUtilization.
5. Click Select metric.

Step 3: Configure the alarm conditions

Enter these settings:

- Statistic: Average
- Period: 5 minutes
- Threshold type: Static
- Condition: Greater than

- Threshold: 85
- Evaluation periods: 2
- Datapoints to alarm: 2 out of 2

Step 4: Finish

1. Click Next.
2. Optionally configure an SNS topic to receive email notifications. If creating a new topic, confirm the subscription email.
3. Name the alarm High-CPU-85-Alarm.
4. Review the settings and click Create alarm

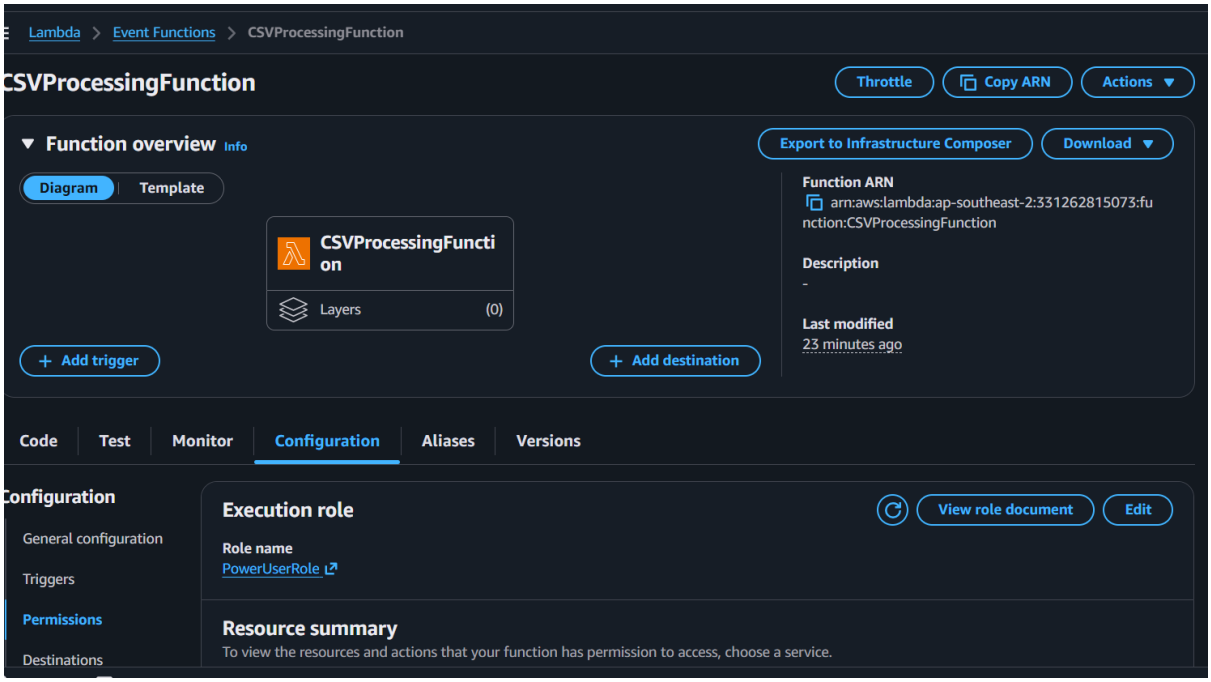
d. Identify one method for investigating the root cause of high CPU utilization. (1 mark)

Use CloudWatch metrics and EC2 system logs.

1. Open CloudWatch and view the instance's CPUUtilization graph.
2. Identify when CPU usage started increasing.
3. Connect to the EC2 instance and use operating-system tools such as top on Linux to identify processes consuming excessive CPU.
4. Check application logs for errors, heavy requests, or unusual activity.

Question 12: Event-Driven Processing (5 Marks) A company uploads CSV files to an S3 bucket. Each uploaded file must be validated and processed automatically, with the results stored in another location. Design a solution using AWS services.

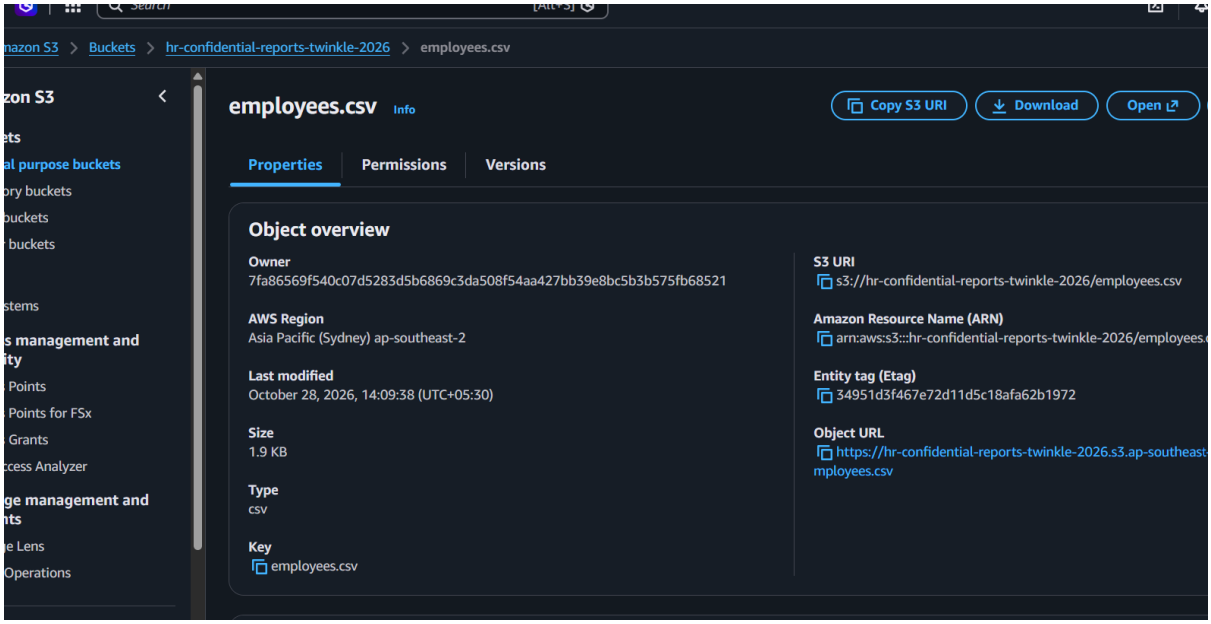
a. Identify a suitable service for executing the processing code. (1 mark)

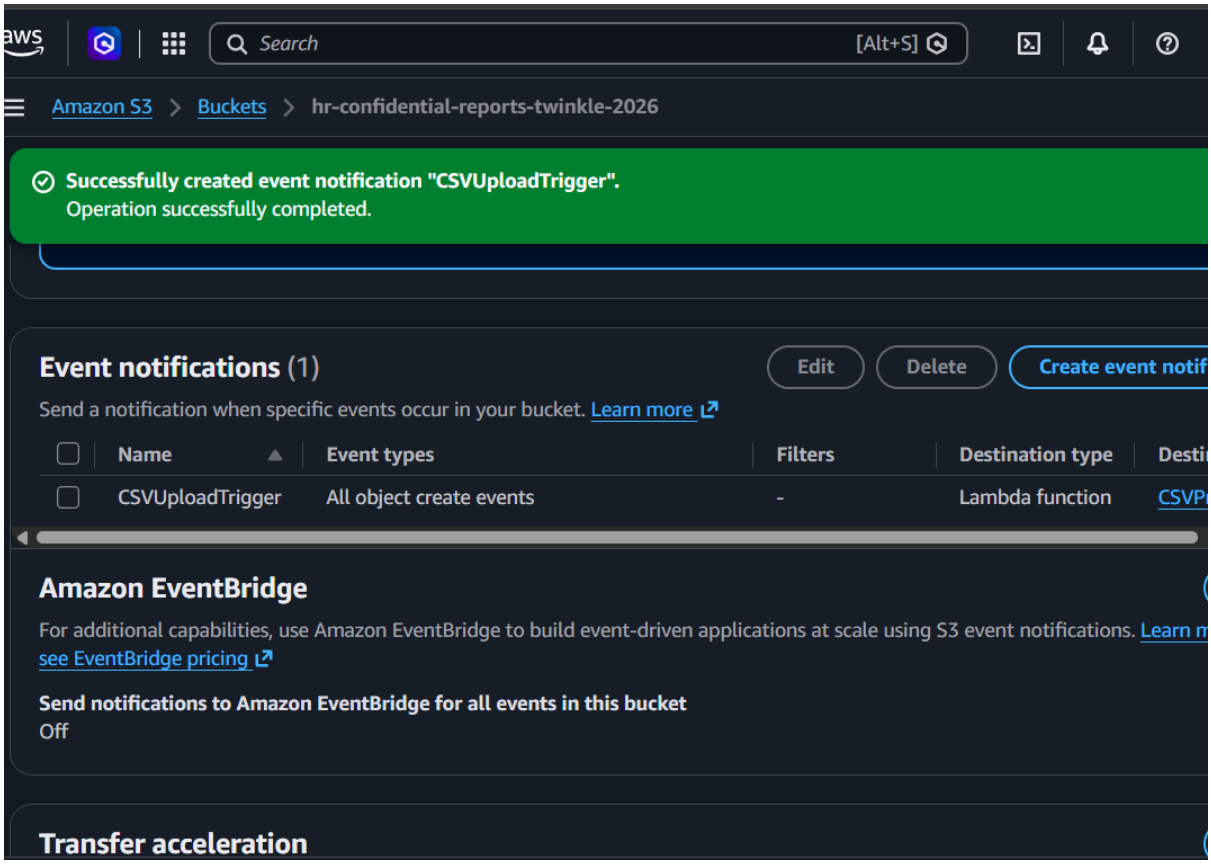


AWS Lambda.

AWS Lambda executes code automatically when triggered by an event. It can read the uploaded CSV file, validate its contents, process the records, and save the results to an output S3 bucket.

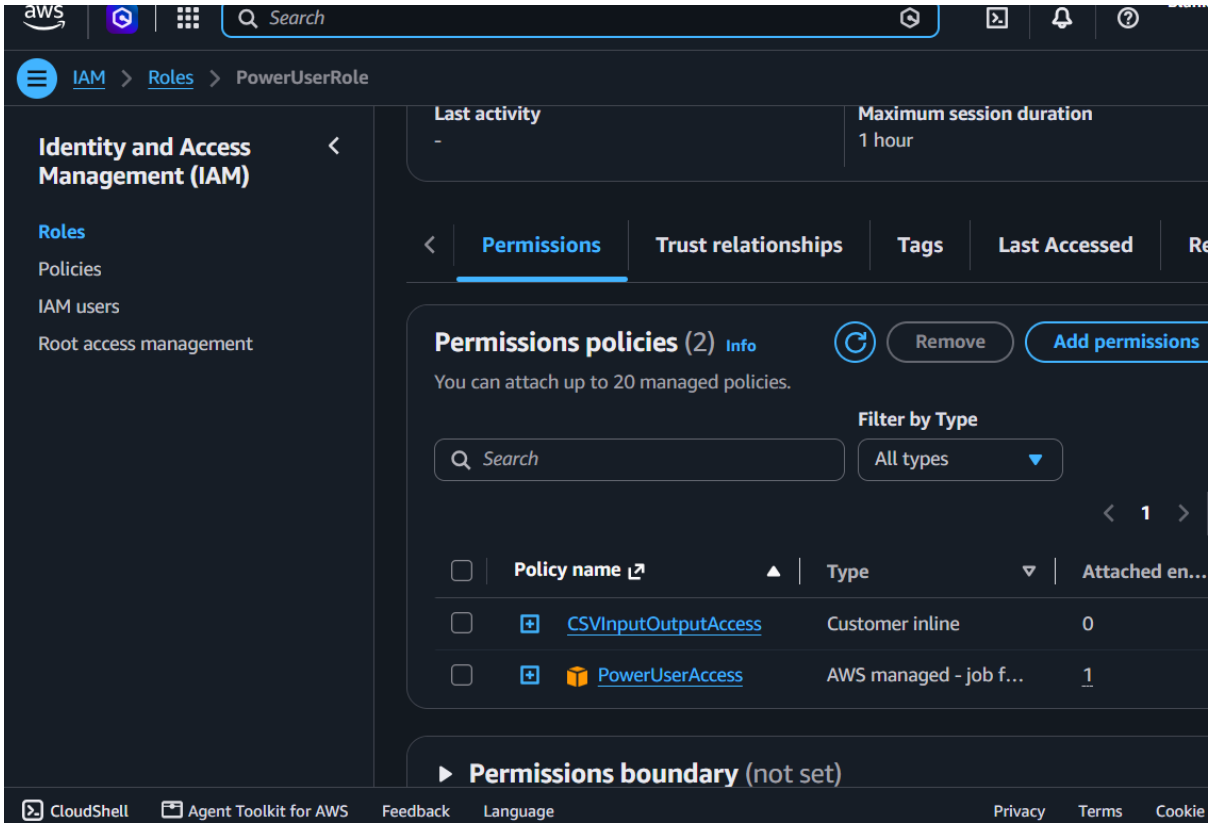
b. Explain how an S3 upload can trigger processing. (1 mark)

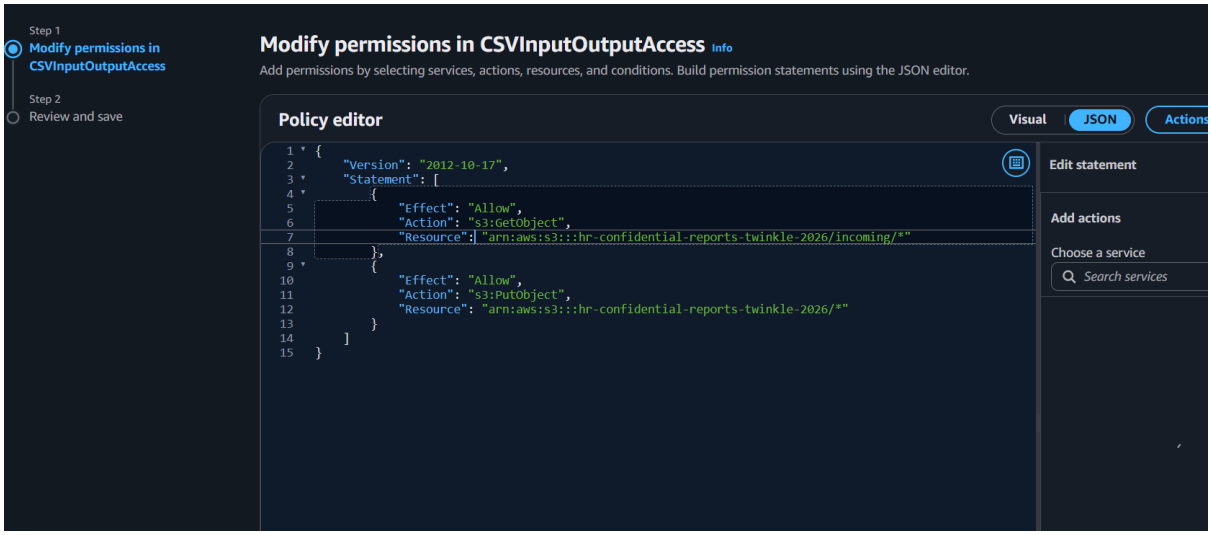




Configure an Amazon S3 Event Notification for the ObjectCreated event. When a CSV file is uploaded to the input bucket, S3 invokes the configured Lambda function, which starts processing the file.

c. Explain the IAM permissions needed to read the input file and write the results. (1 mark)



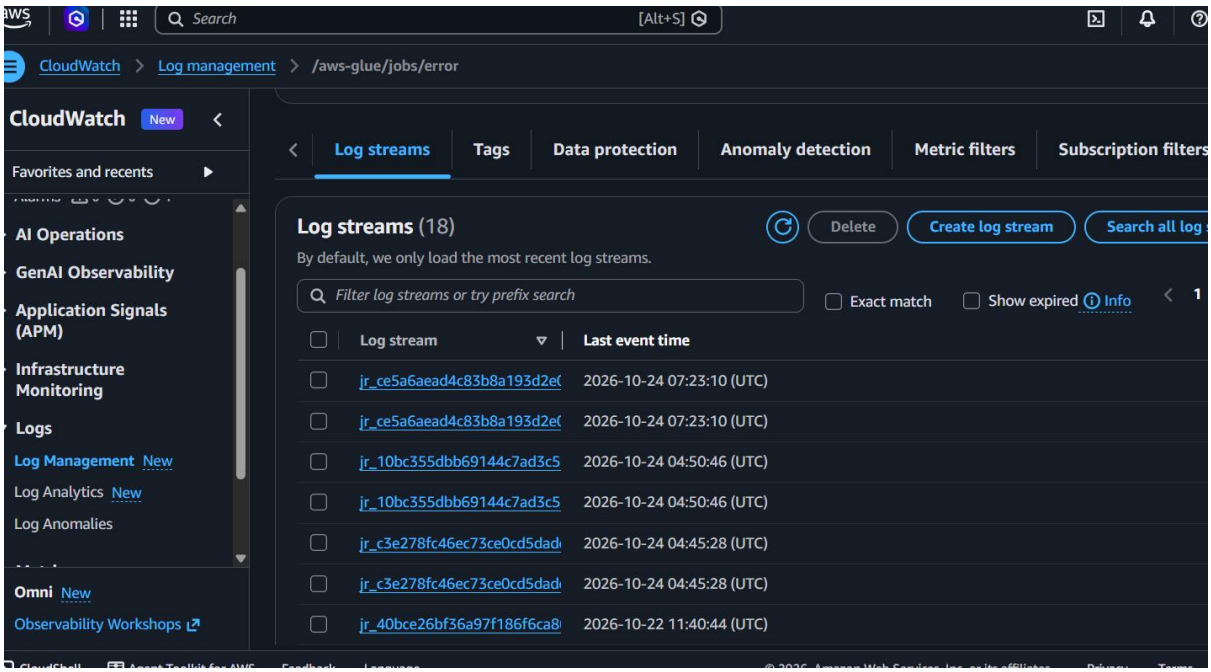


Create an IAM execution role for Lambda with least-privilege permissions:

- s3:GetObject on the input bucket's objects to read CSV files.
- s3:PutObject on the output bucket's objects to save processed results.
- CloudWatch Logs permissions to write execution logs.

The policy should restrict access to the required buckets rather than granting unrestricted S3 access.

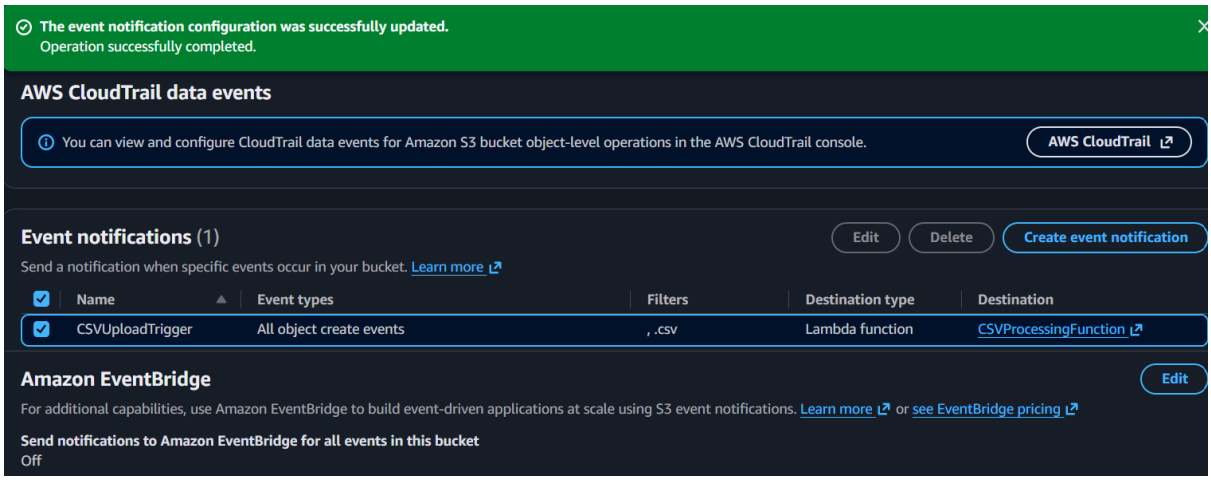
d. Explain how errors and processing logs can be captured. (1 mark)



Use Amazon CloudWatch Logs to capture Lambda execution logs, including processing messages and errors. Configure CloudWatch alarms for important failures if required. For reliable retry handling,

configure a Lambda dead-letter queue or a failure destination to retain failed event information for later investigation.

e. Ex plain how to prevent duplicate processing or recursive triggering. (1 mark)



 Prevent recursive triggering: Use separate input and output buckets, or configure the S3 notification to trigger only for a specific input prefix such as incoming/ and exclude the output prefix.

 Prevent duplicate processing: Implement idempotency by recording each processed object's key and version or unique identifier, so processing the same event again does not create duplicate results.

SECTION E: AWS STEP FUNCTIONS || Maximum Marks: 15 Question

13: State Machine Design (7 Marks) A company needs to process sales files through the following stages:

1. Validate the input file.

2. Transform the data. 3. Calculate total sales by product.

4. Store the results in Amazon S3.

5. Send a completion notification. Design an AWS Step Functions state machine to coordinate the workflow.

a. Identify suitable states for validation, processing, storage, and notification. (2 marks)

1. ValidateFile (Task State): Invokes a Lambda function to check whether the input sales file is valid and contains the required data.
2. CheckValidity (Choice State): Checks the validation result and routes the workflow depending on whether the file is valid or invalid.

3. TransformData (Task State): Invokes a Lambda function to clean and transform the sales data.
4. CalculateSales (Task State): Calculates the total sales for each product.
5. StoreResults (Task State): Saves the processed results in an Amazon S3 bucket.
6. SendNotification (Task State): Uses Amazon SNS or another supported service to send a completion notification.
7. Success (Succeed State): Marks the workflow as completed successfully.
8. InvalidFile (Fail State): Stops the workflow if the input file fails validation.

b. Explain how a Choice state can route invalid files to a failure path and valid files to processing. (1 mark)

The Choice state evaluates the validation result returned by the ValidateFile task. If the file is valid, the workflow proceeds to TransformData. If the file is invalid, it routes to the InvalidFile state and stops processing.

c. Explain how Retry and Catch can handle transient failures and permanent errors. (2 marks)

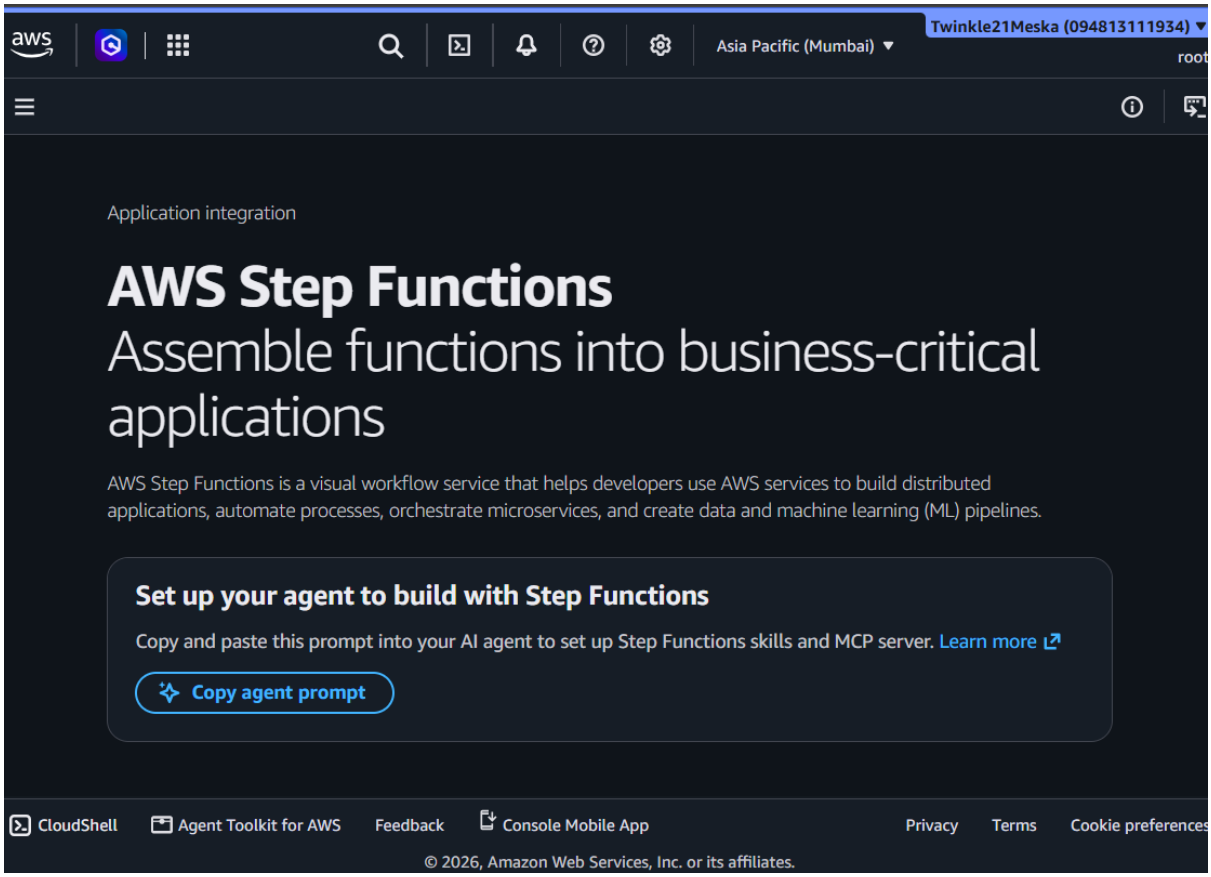
- Retry: Automatically retries a failed task when a temporary error occurs, such as a service throttling error or temporary network issue. The number of attempts and waiting intervals can be configured.
- Catch: Handles errors that remain after retries or other task failures by routing execution to a specified error-handling state. For example, a permanent processing or S3 storage error can be routed to a failure-handling task that records the error and sends an alert.

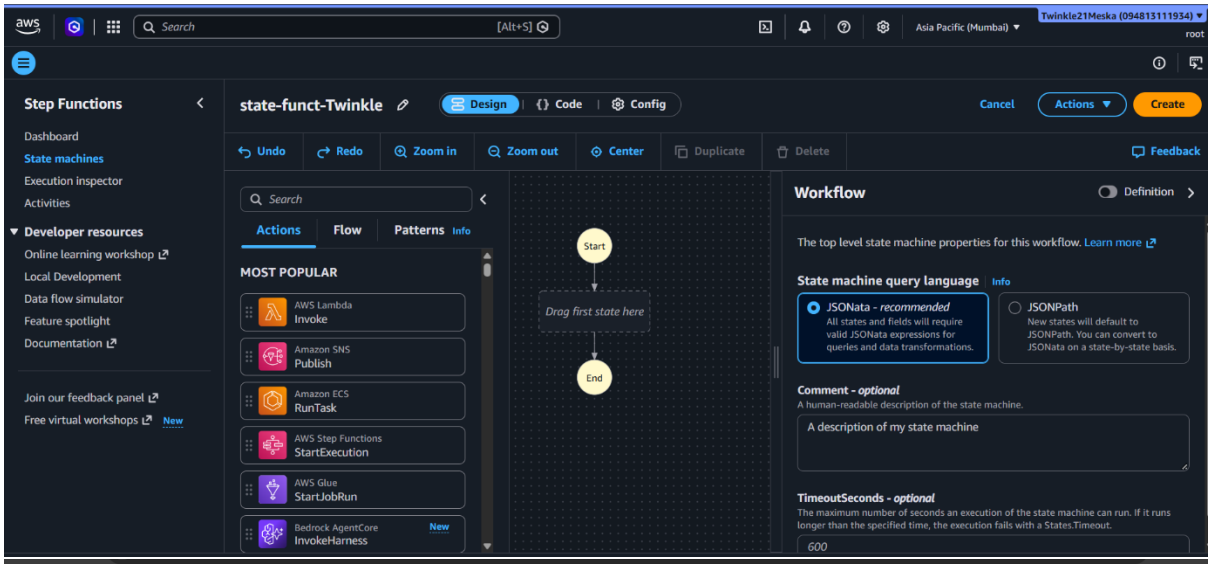
d. Explain how the state machine can invoke Lambda functions or other supported AWS services. (1 mark)

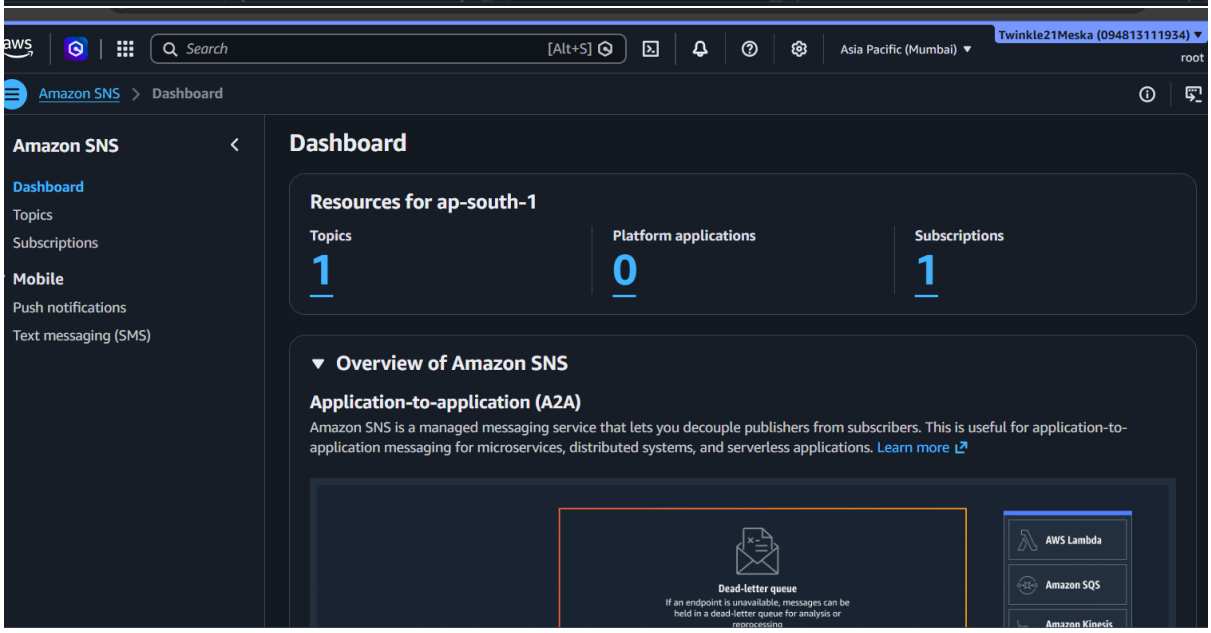
AWS Step Functions can invoke AWS Lambda functions using Task states to validate files, transform data, and calculate product-wise sales totals. It can also integrate with Amazon S3 to store results and Amazon SNS to send notifications. This allows the entire process to run automatically without manual intervention.

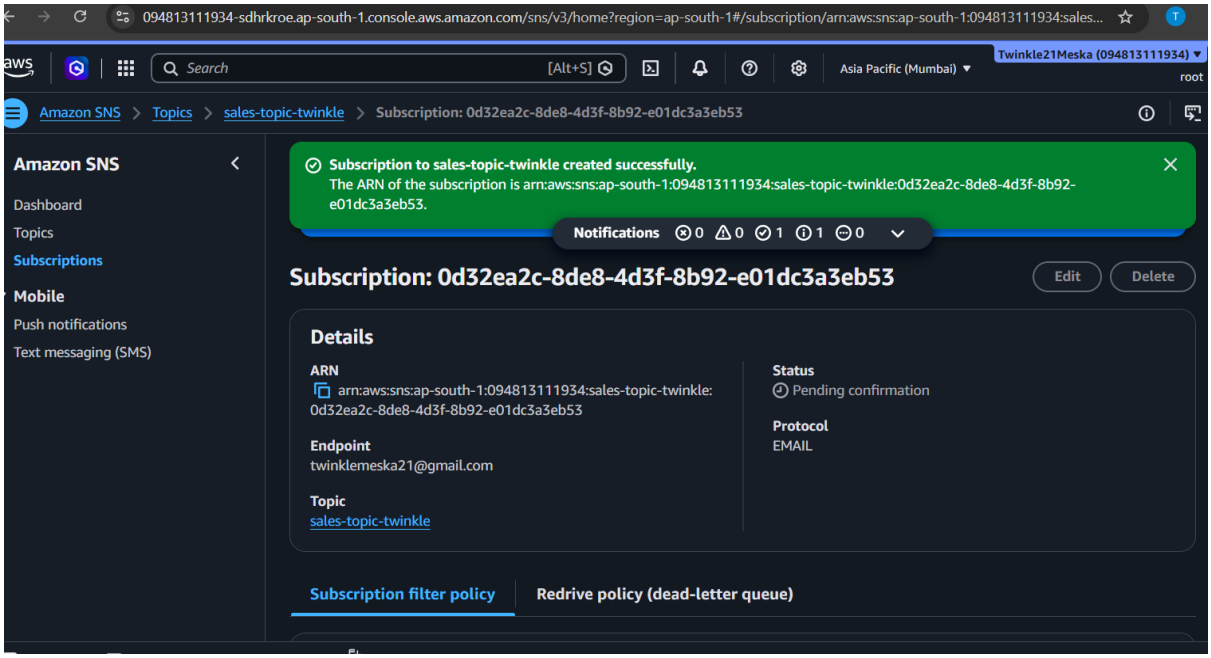
e. Explain how execution history helps troubleshoot a failed workflow. (1 mark) Expected Output: Draw the workflow or submit an Amazon States Language (ASL) definition.

Execution history records the sequence of states executed, their inputs and outputs, execution status, and error details. By examining this history in the AWS Step Functions console, developers can identify the state that failed, understand the cause, and troubleshoot the workflow.


Twinkle21Meska (094813111934)
root
Asia Pacific (Mumbai)
Application integration
AWS Step Functions
Assemble functions into business-critical applications
AWS Step Functions is a visual workflow service that helps developers use AWS services to build distributed applications, automate processes, orchestrate microservices, and create data and machine learning (ML) pipelines.
Set up your agent to build with Step Functions
Copy and paste this prompt into your AI agent to set up Step Functions skills and MCP server. Learn more
Copy agent prompt
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
Privacy
Terms
Cookie preferences
© 2026, Amazon Web Services, Inc. or its affiliates.


Search
[Alt+S]
Asia Pacific (Mumbai)
Twinkle21Meska (094813111934)
root
Step Functions
Dashboard
State machines
Execution inspector
Activities
Developer resources
Online learning workshop
Local Development
Data flow simulator
Feature spotlight
Documentation
Join our feedback panel
Free virtual workshops
New
state-funct-Twinkle
Design
Code
Config
Cancel
Actions
Create
Undo
Redo
Zoom in
Zoom out
Center
Duplicate
Delete
Feedback
Search
Actions
Flow
Patterns
Info
MOST POPULAR
AWS Lambda
Invoke
Amazon SNS
Publish
Amazon ECS
RunTask
AWS Step Functions
StartExecution
AWS Glue
StartJobRun
Bedrock AgentCore
InvokeHarness
New
Start
Drag first state here
End
Workflow
Definition
The top level state machine properties for this workflow. Learn more
State machine query language
Info
JSONata - recommended
All states and fields will require valid JSONata expressions for queries and data transformations.
JSONPath
New states will default to JSONPath. You can convert to JSONata on a state-by-state basis.
Comment - optional
A human-readable description of the state machine.
A description of my state machine
TimeoutSeconds - optional
The maximum number of seconds an execution of the state machine can run. If it runs longer than the specified time, the execution fails with a States.Timeout.
600


Search
[Alt+S]
Asia Pacific (Mumbai)
Twinkle21Meska (094813111934)
root
Amazon SNS
Dashboard
Amazon SNS
Dashboard
Topics
Subscriptions
Mobile
Push notifications
Text messaging (SMS)
Dashboard
Resources for ap-south-1
Topics
1
Platform applications
0
Subscriptions
1
Overview of Amazon SNS
Application-to-application (A2A)
Amazon SNS is a managed messaging service that lets you decouple publishers from subscribers. This is useful for application-to-application messaging for microservices, distributed systems, and serverless applications. Learn more
Dead-letter queue
If an endpoint is unavailable, messages can be held in a dead-letter queue for analysis or reprocessing
AWS Lambda
Amazon SQS
Amazon Kinesis



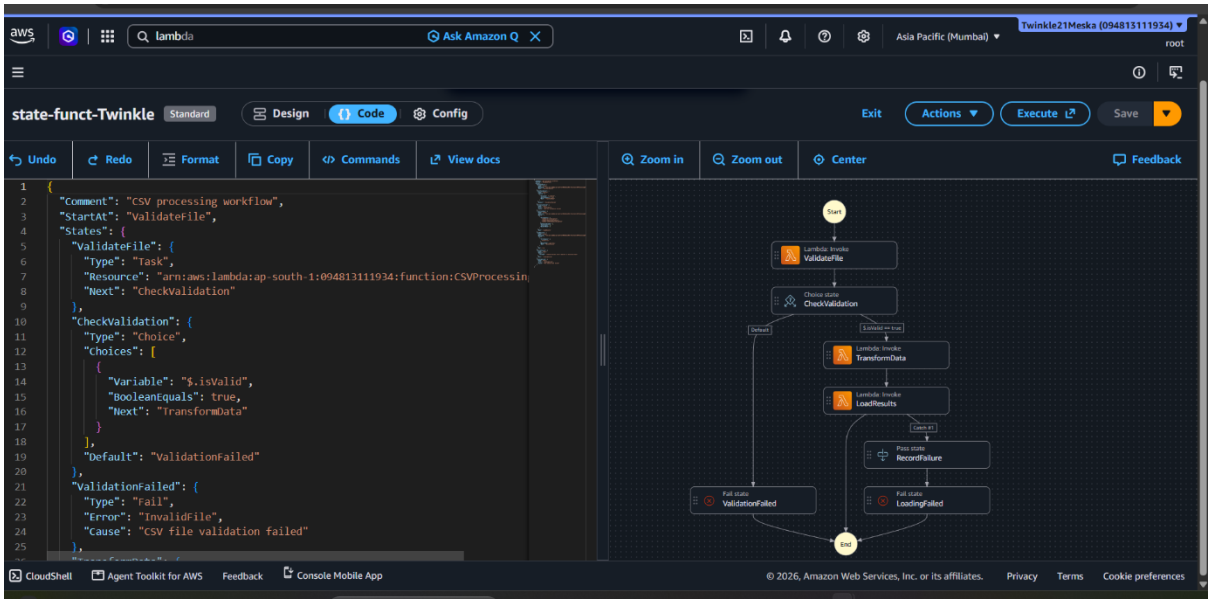
Question 14: Step Functions Implementation (8 Marks) A data engineering team has developed three processing functions: • ValidateFile • TransformData • LoadResults The functions must execute in sequence.

Requirements: • If validation fails, the workflow must stop. • If transformation fails temporarily, the workflow must retry. • If loading fails permanently, the workflow must record the failure. Tasks:

a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary. (3 marks)

The Amazon States Language (ASL) JSON definition creates a state machine in AWS Step Functions with three sequential Task states: ValidateFile, TransformData, and LoadResults.

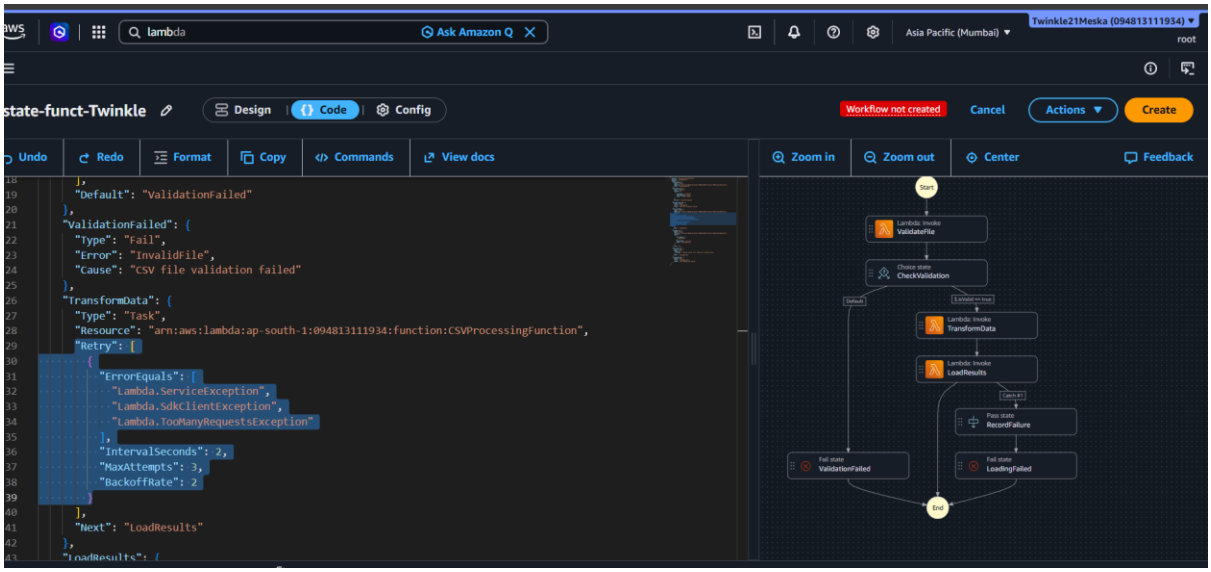
- StartAt: Specifies the first state to execute.
- Task: Invokes the specified AWS Lambda function to perform an operation.
- Next: Defines the next state in the workflow.
- End: Indicates that the workflow has completed.

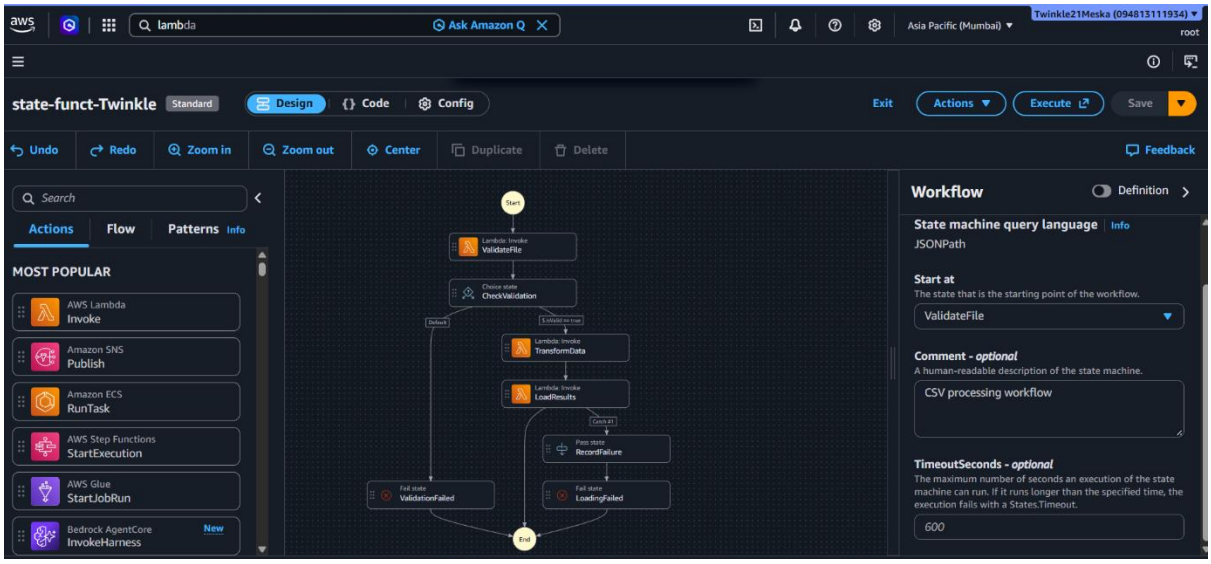


b. Configure a retry policy for the transformation task with an appropriate error category, retry interval, and maximum attempts. (2 marks)

The Retry field is configured in the TransformData Task state to handle temporary errors during data transformation.

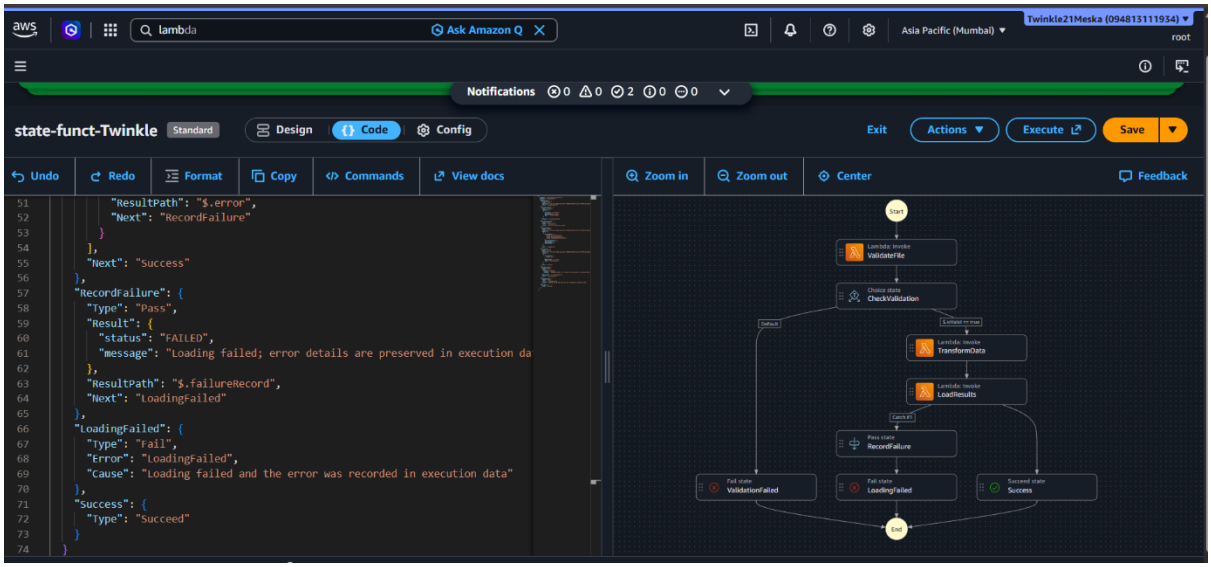
- ErrorEquals: Identifies temporary Lambda service errors, client errors, and throttling errors that should be retried.
- IntervalSeconds: 2: Waits 2 seconds before the first retry.
- MaxAttempts: 3: Allows up to three retry attempts after the initial attempt.
- BackoffRate: 2: Doubles the waiting interval after each retry.





c. Configure a Catch transition for the loading task that routes to an error-handling state. (1 mark)

The Catch field is configured in the LoadResults Task state to handle loading errors. If loading fails, the Catch transition captures the error details and stores them at $.error. The workflow then moves to the RecordFailure state for error handling and finally enters the LoadingFailed state to terminate the execution.



d. Explain how data from one task is passed to the next task. (1 mark)

AWS Step Functions passes the output of one state as the input to the next state by default. In this workflow, the output from ValidateFile is passed to CheckValidation, and if the file is valid, the data is passed to TransformData. The transformed output is then passed to LoadResults.

e. Explain how you would test successful and failed executions. (1 mark) Expected Output: Submit the JSON definition and explain the states, transitions, and error-handling strategy.

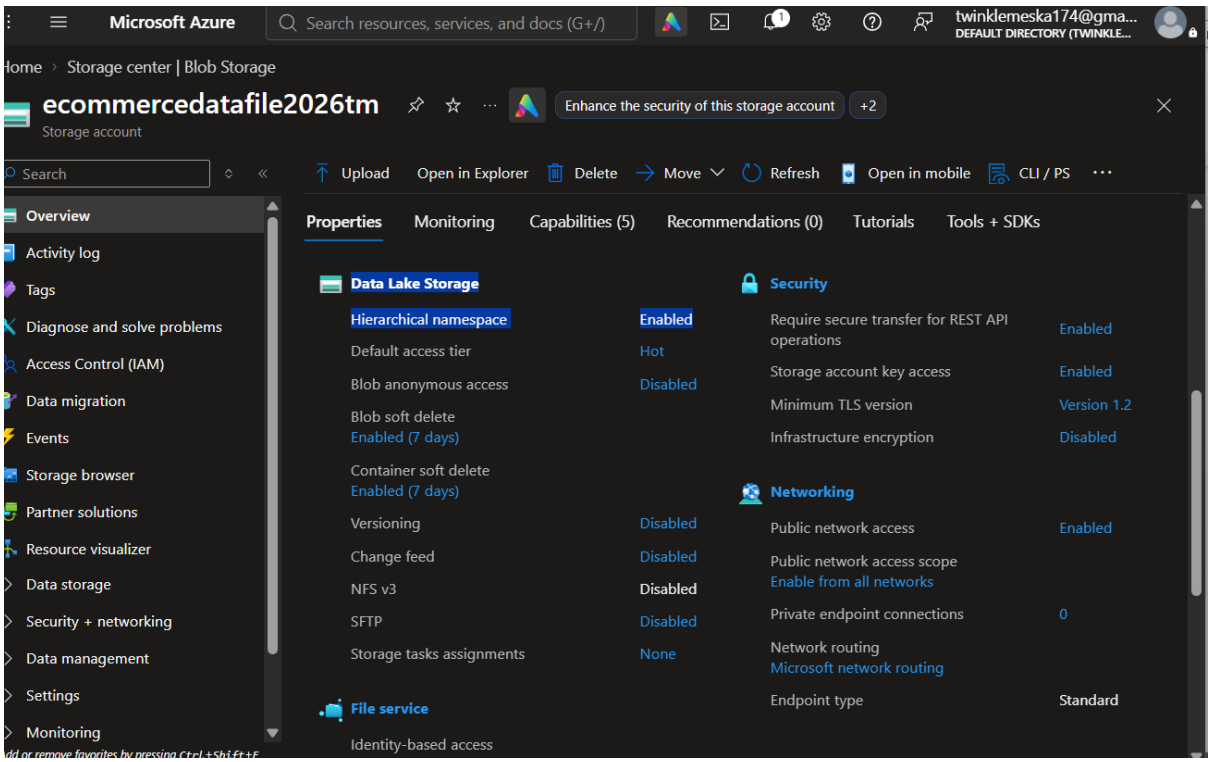
To test a successful execution, start the state machine with valid input data and verify that the workflow completes all three tasks: ValidateFile, TransformData, and LoadResults.

To test a failed execution, provide an invalid file or simulate an error in one of the Lambda functions. Verify that validation failure stops the workflow, temporary transformation errors trigger retries, and loading errors are routed to the RecordFailure state.

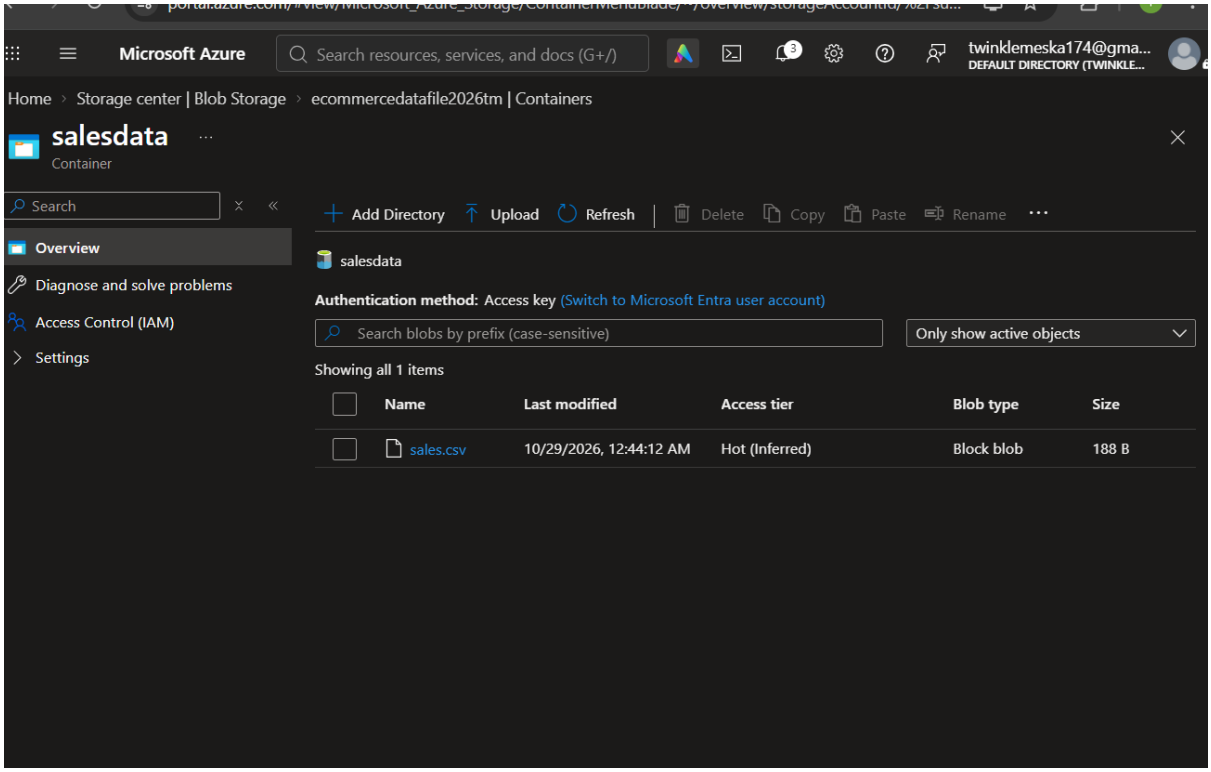
SECTION F: MICROSOFT AZURE || Maximum Marks: 15

Question 15: Azure Data Lake Storage and Security (5 Marks) A company needs to store confidential business data in Azure for analytics. Perform or document the following:

a. Configure or describe a storage account with hierarchical namespace enabled for Azure Data Lake Storage Gen2. (1 mark)



b. Create a filesystem or container and upload a sample dataset. (1 mark)



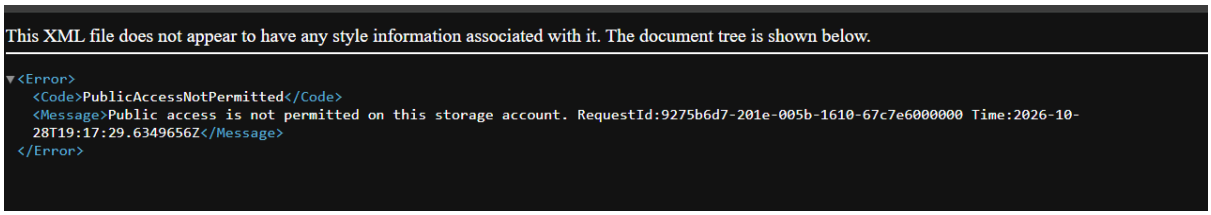
c. Explain how Microsoft Entra ID and Azure role-based access control can manage access to the data. (2 marks)

Microsoft Entra ID manages user identities and authenticates users. Azure role-based access control (RBAC) assigns permissions to users, groups, or applications according to their responsibilities. For example, a data analyst can be assigned the Storage Blob Data Reader role to read the dataset, while an authorized data engineer can receive Storage Blob Data Contributor to read, upload, and modify data. Assigning permissions at the storage account or container scope follows the principle of least privilege

d. Explain how you would verify that unauthorized users cannot access the dataset. (1 mark)

To verify that unauthorized users cannot access the dataset, the storage container should be configured as Private, with anonymous access disabled. Access should be granted only to authenticated users who have the required permissions through Microsoft Entra ID and Azure RBAC.

To test this, copy the URL of the uploaded sales.csv file and open it in an Incognito browser window without signing in. If the configuration is correct, Azure should deny access and display an authorization error, such as ResourceNotFound or AuthorizationFailure, depending on the request and configuration.



Step 1: Configure Azure Data Lake Storage Gen2

Sign in to the Microsoft Azure Portal. Open Storage accounts and create a storage account by selecting your subscription, resource group, unique account name, and region. Under the Advanced tab, enable Hierarchical namespace to support Azure Data Lake Storage Gen2. Review the configuration and create the storage account.

Step 2: Create a Container and Upload a Dataset

Open the newly created storage account and navigate to Data storage → Containers. Create a container named confidential-data and keep anonymous access disabled. Create a sample CSV file named businessdata.csv containing example business records, such as transaction IDs, product names, quantities, and sales amounts. Upload the file to the container and verify that it appears in the file listing.

Step 3: Assign Access Permissions Using Microsoft Entra ID

Open the storage account and select Access control (IAM). Choose Add → Add role assignment. Assign the appropriate Azure role to the intended user or group. For example, use Storage Blob Data Reader for users who only need to view or download data, or Storage Blob Data Contributor for users who need to read, upload, and modify data. Select the authorized identity from Microsoft Entra ID and complete the role assignment.

Step 4: Apply Least-Privilege Access Control

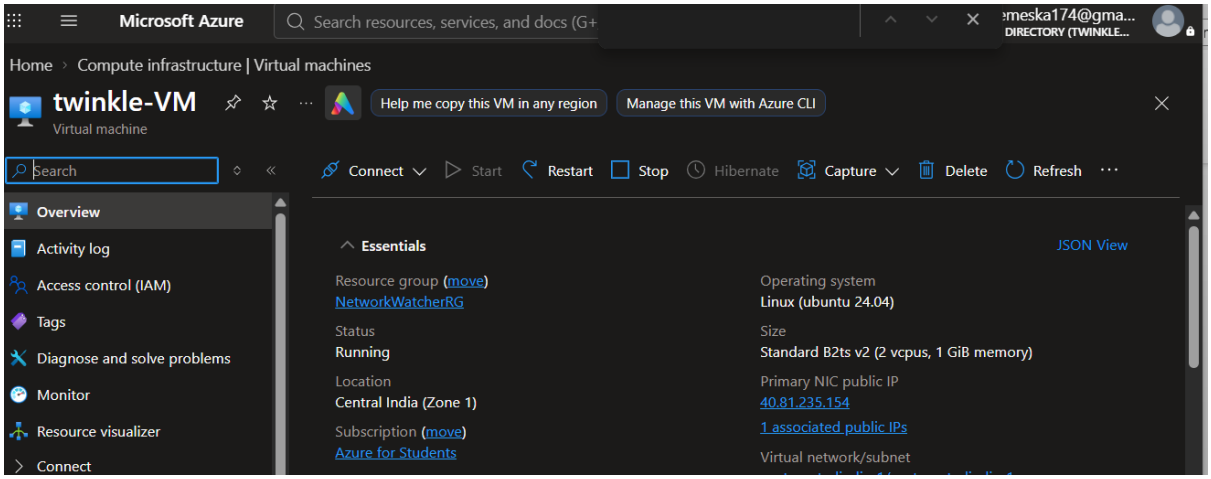
Grant users only the permissions required for their work. For example, analysts who only need to read business datasets should receive read-only data access rather than write or delete permissions. Where appropriate, assign roles at the container scope instead of the entire storage account to limit access. Keep anonymous access disabled and review inherited role assignments to avoid granting unnecessary permissions.

Step 5: Verify Authorized and Unauthorized Access

Sign in using an authorized test identity and attempt to access businessdata.csv. Confirm that the assigned permissions allow the expected operations. Next, use a separate test identity that has no applicable data-access role and attempt to access the same file through an authenticated client or Azure Storage Explorer. Confirm that access is denied. Review the identity's inherited permissions and ensure the container has no public access.

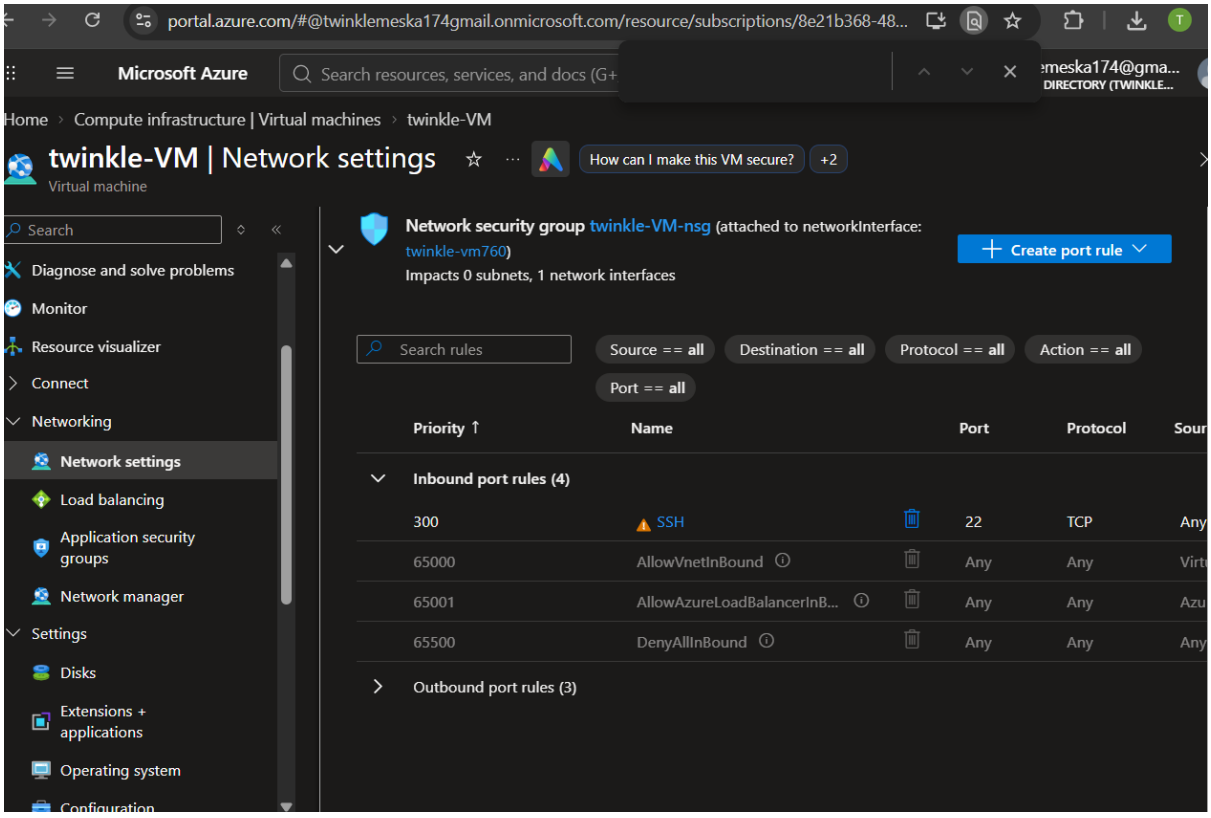
16: Azure VM Troubleshooting (5 Marks) A virtual machine is running in Azure, but administrators cannot connect to it. Perform or document the following:

a. Verify the VM's provisioning and power states. (1 mark)



b. Explain how Network Security Group rules can prevent connectivity. (1 mark)

A Network Security Group (NSG) controls incoming and outgoing network traffic using security rules. If an inbound rule denies SSH traffic on port 22, or if no suitable allow rule exists before a blocking rule, administrators cannot connect to a Linux virtual machine. The source IP address, destination port, protocol, and rule priority must be checked to identify connectivity issues. For security, SSH access should be allowed only from trusted IP addresses rather than from all internet addresses.



c. Explain how subnet and routing configuration affect connectivity. (1 mark)

The virtual machine must be connected to a correctly configured virtual network and subnet. Incorrect subnet settings, route tables, or next-hop configurations can prevent network traffic from reaching the VM or returning to the administrator. To troubleshoot connectivity, administrators should verify the VM's subnet, network interface, effective routes, and network connectivity between the source and destination.

d. Recommend a secure troubleshooting method that avoids unnecessarily exposing the VM to the public internet. (2 marks)

Azure Bastion is a secure way to connect to an Azure virtual machine through the Azure portal without requiring a public IP address on the VM or exposing SSH/RDP ports directly to the internet. Administrators can also use Azure Network Watcher to diagnose connectivity problems, check Network Security Group (NSG) rules, and analyze network routes.

If Azure Bastion is unavailable, administrators can use a private VPN connection to access the VM. SSH access should be restricted to trusted IP addresses, and unnecessary inbound ports should remain closed. These measures reduce the risk of unauthorized access and protect the vir

Step 1: Verify VM Provisioning and Power State

Open the Azure Portal and search for Virtual Machines. Select the required VM and check its Provisioning state and Power state under the Overview section. Ensure that provisioning is successful and the VM is running. If the VM is stopped or provisioning has failed, resolve the issue before testing connectivity.

Step 2: Check Network Security Group (NSG) Rules

Open the VM's Networking settings and review the associated Network Security Group (NSG). Check the inbound and outbound security rules to ensure that the required traffic is allowed. For example, SSH uses TCP port 22 for Linux VMs, while RDP uses TCP port 3389 for Windows VMs. Incorrect or higher-priority deny rules can block connectivity. Allow access only from trusted IP addresses.

Step 3: Verify Subnet and Routing Configuration

Check the virtual network and subnet assigned to the VM. Verify that the subnet has the correct address range and that the VM's network interface is attached to the expected subnet. Review the route table and effective routes to ensure traffic can reach its destination. Incorrect routes, missing return routes, or routing through an unavailable network appliance can prevent connectivity.

Step 4: Use Azure Network Troubleshooting Tools

Use Azure Network Watcher to diagnose connectivity problems. Check effective security rules and effective routes for the VM's network interface. Use IP flow verification or connection troubleshooting, where supported, to identify whether traffic is blocked by an NSG or routing configuration. Correct the identified issue and test the connection again.

Step 5: Follow Secure Troubleshooting Practices

Avoid exposing SSH or RDP ports directly to the public internet. Use Azure Bastion to connect securely to the VM through the Azure Portal without requiring a public IP address on the VM. Alternatively, use a VPN or a private network connection. Restrict access to trusted sources, apply least-privilege permissions, and enable monitoring and logging to help detect future connectivity problems.

Question 17: Azure Data Integration (5 Marks) A company receives a CSV file containing daily customer transactions and needs to prepare it for analysis. Design a solution using Azure services.

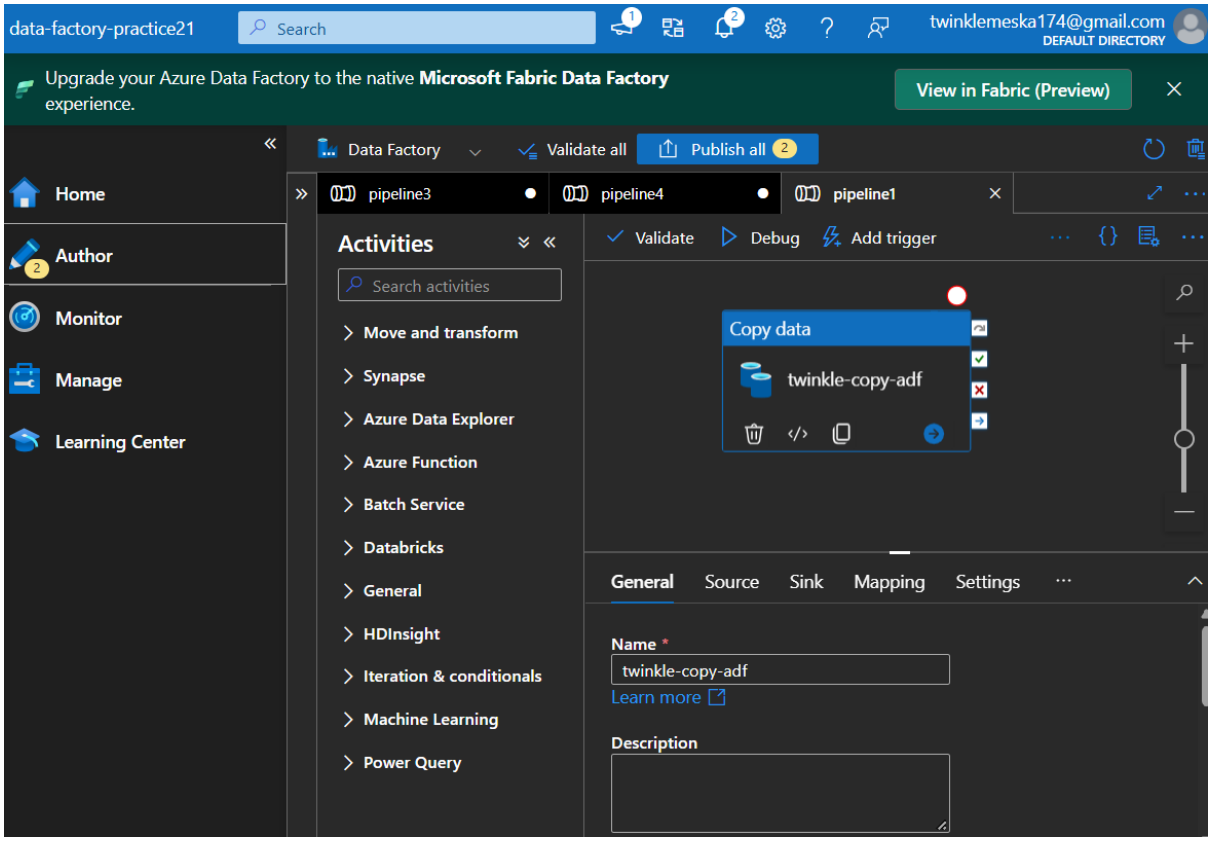
a. Select an appropriate service to orchestrate data movement. (1 mark)

b. Explain how to copy the CSV file into Azure Data Lake Storage Gen2. (1 mark)

c. Describe how to organize files by ingestion date. (1 mark)

d. Explain how to handle malformed records and missing values. (1 mark)

e. Explain how to monitor pipeline execution and validate data completeness. (1 mark)



Step 1: Create Azure Data Factory

Open the Azure Portal and search for Data Factory. Click Create and select your subscription, resource group, region, and a unique Data Factory name. Click Review + create, then Create. After deployment, open Azure Data Factory Studio to build and manage your data pipeline.

Step 2: Copy the CSV File into Azure Data Lake Storage Gen2

Create a storage account with hierarchical namespace enabled to use Azure Data Lake Storage Gen2. Create a container to store the transaction files. In Azure Data Factory Studio, create linked services for the source CSV location and the ADLS Gen2 storage account. Create a source dataset for the CSV file and a sink dataset for the destination. Add a Copy Data activity to the pipeline, configure the source and destination datasets, and run the pipeline to copy the file.

Step 3: Organize Files by Ingestion Date

Configure the destination path to use date-based folders, such as year=2026/month=10/day=28. Use dynamic expressions in the sink dataset or pipeline parameters to generate the folder path based on

the ingestion date. This structure organizes daily transaction files and makes it easier to retrieve data for a specific date.

Step 4: Handle Malformed Records and Missing Values

Validate the CSV data for missing values, invalid dates, incorrect data types, and missing required fields. Use Azure Data Factory mapping data flows with validation and conditional split transformations, or use Azure Databricks with PySpark for more complex cleaning. Store valid records in the processed folder and invalid records in a separate quarantine folder for further investigation.

Step 5: Monitor Pipeline Execution and Validate Data Completeness

Open the Monitor section in Azure Data Factory Studio to check pipeline status, activity runs, execution duration, and error messages. Configure alerts for pipeline failures. Validate data completeness by comparing source and destination record counts, checking required fields, and confirming that the expected files exist in the correct date-based folders. Record the number of valid and rejected records to support data quality checks.